

DD: BIL-02 — Invoicing (Core)

Developer implementation guide: DD_BIL-02-Invoicing-Core-DEV_IMPL_GUIDE-v2.11.md.

1. Purpose

This rewritten DD is the developer-facing version of the module contract DD_BIL-02-Invoicing-Core-v2.11.md.

Verdict: ■ New | **Wave:** 1 | **Group II:** Billing & Revenue | **Version:** v2.11 (core) BIL-02 Core owns the invoice entity, lifecycle, type catalog, sequence allocation, per-operator configuration, multi-tax summary, customer snapshot, and grouping policy. The five satellite DDs each handle one specific capability layered on top: generation, tax invoice signing, adjustments, read API, and status management. Each satellite is independently reviewable. **Satellite DDs:** BIL-02-GEN-01 Invoice Generation, BIL-02-TAX-01 Tax Invoice & Gateway, BIL-02-ADJ-01 Invoice Adjustments, BIL-02-READ-01 Invoice Read API, BIL-02-STA-01 Status Management & Recovery. **Version history:** see CHANGELOG_BIL-02-Invoicing-Core.md in the same directory for the full evolution from v1.0 through v2.11.

The goal of this rewrite is to make the DD easier for a mid-level developer to implement without losing the detailed source contract.

2. Implementation Scope

This DD should be implemented as part of the owning SOPHIX capability, not as an isolated service unless the deployment architecture explicitly separates that capability.

Implement this DD with these rules:

- Use the owning module APIs/repositories for authoritative writes.
- Use approved internal APIs/client wrappers when reading another module's data.
- Do not query another module's database tables directly from business flow code.
- Treat worker names as logical Camunda external-task topics, not separate deployments.
- One worker application may handle multiple topics, but metrics, retries, and logs must remain visible per topic.
- Emit success events only after the authoritative state commit succeeds.
- Use outbox publishing for Kafka events.

3. Runtime Metadata

Item	Value
Source file	/Users/macbook/Downloads/Sophix_V3_BSS_DDs_MD_2026-05-27/05_billing/DD_BIL-02-Invoicing-Core-v2.11.md
Version	v2.11
DD type	module contract
BPMN/process keys	Not explicitly defined
Drools packages	Not explicitly defined

4. Runtime Contracts To Implement

4.1 APIs

- GET /api/customers/me/documents
- GET /api/invoice-sequence-gaps
- GET /api/invoice-types
- GET /api/invoice-types/{typeCode}/operator-config/{op}
- GET /api/invoices/by-legal-number/{legalNumber}
- GET /api/invoices/{id}

- GET /api/invoices?account_id={id}
- GET /api/invoices?customer_id={id}
- GET /api/invoices?operator={code}&status={s}
- GET /api/invoices?subscription_id={id}
- GET /api/proforma/{id}
- GET /api/proforma?subscription_id={id}
- POST /api/billing/charges/compute
- POST /api/invoice-sequence-gaps/{id}/resolve
- POST /api/invoice-types
- POST /api/invoice-types/{typeCode}/operator-config

4.2 Tables

- No CREATE TABLE block is explicitly declared in this DD.

For each table in this DD, implement the schema with:

- a short table comment explaining what the table is for;
- column comments or migration notes explaining each field;
- constraints and indexes from the DD;
- seed data where the DD provides operator-specific sample rows.

4.3 Worker Topics

- billing-engine
- billing-invoicing
- put-master-fields

Worker-topic guidance:

- A BPMN service task uses the topic.
- A worker application subscribes to the topic.
- The topic handler validates input variables, calls the owning module/API, writes outputs back to Camunda, and records metrics.
- Do not treat the topic string as a shell command or Kubernetes deployment name.

4.4 Events

- InvoiceCancelled
- InvoiceEventPublisher
- InvoiceGenerationContext
- InvoiceGenerationService
- InvoiceGenerator
- InvoiceGeneratorRegistry
- InvoiceIssued
- InvoiceLineBuilder
- InvoiceOverdue
- InvoicePaymentApplied
- InvoicePresented
- InvoiceSequenceGap

- InvoiceSequenceService
- InvoiceSkippedDueToPendingTooLong
- InvoiceWriter
- PaymentApplied
- PaymentAppliedConsumer

Event guidance:

- Write events through the transactional outbox.
- Include operation id, aggregate id, operator code, actor, and correlation data.
- Consumers should be able to rebuild downstream state without querying workflow internals.

5. Developer Implementation Checklist

1. Add or verify database migrations and seed rows.
2. Add or verify config rows for the operator/module.
3. Implement request/command DTOs and validation.
4. Implement idempotency and in-flight operation checks where the DD requires them.
5. Implement Drools fact mapping and package deployment where rules are used.
6. Implement BPMN process, service tasks, gateways, timers, and message catches where the DD is a flow.
7. Implement worker-topic handlers using owning module APIs.
8. Implement compensation paths and manual recovery paths.
9. Emit success/rejected events through the outbox.
10. Add smoke tests for happy path, validation failure, dependency failure, and retry/idempotency.

6. Infrastructure And AWS Notes

Deploy this DD with the existing SOPHIX platform components unless the owning module requires isolation:

Concern	Recommendation
API/runtime	Run inside the owning module deployment or existing workflow API pods.
Workers	Consolidate low-volume topics into shared worker apps; keep per-topic metrics.
Database	Use the owning module PostgreSQL schema and migration pipeline.
Kafka	Use the foundation Kafka/MSK setup and transactional outbox.
Camunda	Deploy BPMN files through the workflow deployment pipeline.
Drools	Deploy KJAR/rule artifacts through the approved rules pipeline.
Secrets	Use the platform secret manager; on AWS prefer Secrets Manager/Parameter Store with IRSA.
Observability	Alert on stuck operations, failed workers, outbox backlog, and external dependency failures.

7. Original DD Detail Preserved

The following section preserves the detailed source contract for implementation and review.

Verdict: ■ New | **Wave:** 1 | **Group II:** Billing & Revenue | **Version:** v2.11 (core)

BIL-02 Core owns the invoice entity, lifecycle, type catalog, sequence allocation, per-operator configuration, multi-tax summary, customer snapshot, and grouping policy. The five satellite DDs each handle one specific capability layered on top: generation, tax invoice signing, adjustments, read API, and status management. Each satellite is independently reviewable.

Satellite DDs: BIL-02-GEN-01 Invoice Generation, BIL-02-TAX-01 Tax Invoice & Gateway, BIL-02-ADJ-01 Invoice Adjustments, BIL-02-READ-01 Invoice Read API, BIL-02-STA-01 Status Management & Recovery.

Version history: see `CHANGELOG_BIL-02-Invoicing-Core.md` in the same directory for the full evolution from v1.0 through v2.11.

Glossary

Plain definitions for terms used across BIL-02 and satellite DDs.

Term	Meaning
Invoice	Customer-facing bill listing what was billed, for which period, the amount due, and the payment deadline. May have tax authority weight depending on deployment.
Pro forma	Informational preview of the next invoice. Not a legal/tax document. Tells the customer "this is what you'll owe next cycle so you can prepare to pay." Handled by BIL-02-GEN-01.
Tax invoice	A signed proof-of-payment document required by tax authorities in some deployments. Generated after a payment is received. Handled by BIL-02-TAX-01.
Gap-free numbering	Legal invoices are numbered sequentially. The system must never silently skip a number. If issuance fails after a number is reserved, the gap is recorded in an audit table for reconciliation with the tax authority — never silently lost.
Change history table	A parallel table that records every change to a record (status change, payment applied, cancellation, etc.). Each row in the history table captures: who made the change, when, what the field value was before, what it is after. The history is append-only — rows are never edited or deleted. Used by compliance auditors, operations, and customer care to reconstruct exactly what happened to an invoice. Named with <code>_history</code> suffix (e.g., <code>invoice_history</code>).
Per-operator configuration	Wananchi operates in 3 countries (KE/UG/TZ), Yas in Senegal, plus future deployments. Each may need different invoice formats, due-date grace periods, tax-gateway endpoints, etc. Settings live in <code>invoice_type_operator_config</code> .
Wallet type	Operators define their wallet structure via the PLM-CFG-03 Wallet Configuration catalog. Each operator may have one or multiple wallet types. Wallet type codes are operator-configurable, not hardcoded.
Grouping policy	Per-operator config that defines how charges are distributed across invoices at cycle close. The policy has a dimension (WALLET, PACKAGE, SERVICE_CATEGORY, or SINGLE) and a list of groups . Each group lists which keys belong together (e.g., a wallet group lists wallet types; a package group lists package refs). Number of invoices generated per cycle per customer = number of groups in the policy that have non-zero charges. The dimension lets operators decide whether grouping is by wallet, by package, by service category, or none (single combined invoice).
Customer snapshot	A JSONB object stored on the invoice (and pro forma) at issuance, capturing customer name, billing address, email, phone, language preference, tax identifier, and customer type. The snapshot is immutable for that invoice — if the customer's address changes 6 months later, the historical invoice still shows the address as it was at issuance. Used for legal compliance (a 2026 tax invoice must show the customer info as of 2026, not as of when it's re-rendered) and for self-contained PDF rendering by the notification service (no need to call CRM at render time).
Summary vs detailed view	An invoice has a summary page showing top-level totals per service grouping and a detailed page (or pages) showing per-package consumption breakdown.
Billing mode	When the customer pays relative to their service cycle. Two modes: POSTPAID (customer pays AFTER cycle close; receives a regular invoice with a due date) and PREPAID (customer pays BEFORE cycle close; receives a pro forma in advance, and at cycle close the cycle is settled).
POSTPAID	Customer uses service during the cycle, gets a regular invoice at cycle close, has a grace period (typically 14 days) to pay. If unpaid by the due date, the invoice goes OVERDUE and dunning starts. Example operator: Wananchi POSTPAID customers.
PREPAID	Customer pays before the cycle begins. The system issues a pro forma in advance (default 5 days before cycle end) telling the customer how much will be settled. At cycle close, the operator settles the cycle — depending on the operator's PLM-CFG-03 Wallet Configuration, this is either a debit from the customer's wallet (Wananchi-style: customer tops up wallet, system debits at cycle) or a direct per-cycle payment (Yas SN-style: customer pays the cycle amount as a standalone transaction). BIL-02 doesn't care which — it just knows the customer pays before cycle. The wallet-vs-direct-pay choice is owned by BIL-05 Wallet and Topup and is invisible to BIL-02 Core.

Term	Meaning
MACD	Move/Add/Change/Disconnect — one-off charges applied mid-cycle for changes to a customer's subscription. Generates a one-off invoice via BIL-02-GEN-01.

A — Target

1. What this process is about

BIL-02 core owns the **invoice entity** and the **shared infrastructure** that all 5 satellite DDs depend on: the data model, the type catalog, the lifecycle state machine, sequence number allocation, per-operator configuration, multi-tax summary structure, wallet isolation strategy, the REST + Kafka contract, and the scanner framework that satellite scanners plug into.

Core does NOT generate invoices itself — that's BIL-02-GEN-01. Core does NOT issue tax invoices — that's BIL-02-TAX-01. Core does NOT handle adjustments — that's BIL-02-ADJ-01. Core does NOT serve PDFs or the read API — that's BIL-02-READ-01. Core does NOT run the overdue scanner or PTP/dispute logic — that's BIL-02-STA-01.

Core provides the chassis. The satellites bolt on.

What core owns

- **Invoice entity:** the invoice table, immutability rules, lifecycle states, and transitions.
- **Pro forma entity:** the pro_forma table (separate from invoice for legal-compliance query separation).
- **Line items:** the shared invoice_line_item table with two-level (summary + detail) structure. Each line item references a wallet_type_code (FK to PLM-CFG-03 wallet type catalog) — operator-configurable.
- **Type catalog:** invoice_type_definition + invoice_type_operator_config for extensibility (new types added by config + a generator class).
- **Sequence allocation:** gap-free per-(operator, fiscal_year, type) for legal invoices; non-gap-free for pro forma.
- **Per-operator config:** wallet isolation strategy, formats, grace days, fiscal year start, etc. Number and types of wallets per operator come from PLM-CFG-03 (separate catalog).
- **Multi-tax summary:** structured tax_summary JSONB at invoice level for arbitrary tax stacks.
- **Scanner framework:** pattern + audit table for the daily scanners that satellites implement (overdue, pre-due, pro forma, tax retry).
- **REST contract:** read endpoints exposed to admin console and customer portal. Satellites add their endpoints.
- **Kafka contract:** invoice lifecycle events emitted on transitions. Satellites consume + emit their own.
- **Subscription state interaction:** gating on PENDING_* Subscriptions (skip + retry), reading master fields for invoice generation, writing pro forma ID back.

Out of scope (handled in satellite DDs)

- **Invoice generation logic** (which charges to include, how to compute, idempotency on triggers) → BIL-02-GEN-01
- **Tax invoice signing flow** → BIL-02-TAX-01
- **Credit/debit notes and adjustments** → BIL-02-ADJ-01
- **PDF retrieval, caching, read API** → BIL-02-READ-01
- **Overdue scanner, pre-due reminder, PTP, dispute, late fee** → BIL-02-STA-01

Out of scope entirely

- **Charging logic and amount computation** — BIL-01
- **Tax computation rules** — PLM-CFG-02 Tax Configuration
- **Payment recording** — BIL-01
- **PDF rendering, channel dispatch, template management** — DD_NOT-01 Notification Service
- **Dunning state machine** — BIL-04
- **Refund execution (outbound payouts)** — V3.2 / payout module

2. Business rules

Rules grouped by scope: T for type catalog (extensibility), E for entity structure, L for lifecycle, N for numbering, W for wallet isolation, X for multi-tax summary, S for state interactions.

Type catalog rules

ID	Rule	Triggered on
R-BIL-02-T-1	All invoice document types are registered in <code>invoice_type_definition</code> . Each row: <code>type_code</code> (PK), <code>display_name</code> , <code>legal_nature</code> (LEGAL_INVOICE / PRO_FORMA / TAX_INVOICE / ADJUSTMENT_NOTE), <code>generator_class_ref</code> (class name implementing InvoiceGenerator interface — owned by BIL-02-GEN-01 and others), <code>trigger_type</code> (EVENT_DRIVEN / SCHEDULED_SCANNER / ADMIN_ACTION), <code>requires_signing</code> (boolean), <code>legal_sequence_required</code> (boolean), <code>customer_deliverable</code> (boolean), <code>active</code> (boolean).	Catalog schema
R-BIL-02-T-2	V3 seeds these types: CYCLE_POSTPAID, ONE_OFF_INVOICE, PRO_FORMA_CYCLE, TAX_INVOICE, CREDIT_NOTE, DEBIT_NOTE. Each customer-facing document carries its <code>type_code</code> so the type's behavior rules (signing, numbering, lifecycle, due date) are looked up at runtime.	Initial seed
R-BIL-02-T-3	The catalog is operator-aware via <code>invoice_type_operator_config</code> junction. Per (operator_code, invoice_type_code): enabled, signing endpoint + impl, due date grace days, pre-due reminder window, pre-cycle pro forma window, legal number format, fiscal year start month. Admin console exposes these settings.	Per-operator activation
R-BIL-02-T-4	A future deployment adds a new type by: (a) implementing a class implementing InvoiceGenerator; (b) inserting a row in <code>invoice_type_definition</code> ; (c) inserting per-operator config rows. No changes to invoice table schema, existing generators, or event pipeline.	Extensibility

Invoice entity rules

ID	Rule	Triggered on
R-BIL-02-E-1	A legal invoice (LEGAL_INVOICE, TAX_INVOICE, CREDIT_NOTE, DEBIT_NOTE) has: id (ULID), invoice_type_code (FK to catalog), legal_invoice_number (per numbering rules), subscription_id, account_id, customer_id, operator_code, currency, billing_mode (POSTPAID / PREPAID — captured at issuance as immutable historical state per R-C-2), grouping_config_ref (FK to invoice_grouping_config.id — identifies which group this invoice represents), grouping_dimension (denormalized from config — WALLET / PACKAGE / SERVICE_CATEGORY / SINGLE), grouping_key_values (JSONB array of the keys this invoice covers — e.g., ["WALLET_INTERNET", "WALLET_TV"] for a WALLET-dimension group), wallet_type_code (set when grouping_dimension=WALLET and single wallet; otherwise NULL), package_ref (set when grouping_dimension=PACKAGE and single package; otherwise NULL), service_category_code (set when grouping_dimension=SERVICE_CATEGORY and single category; otherwise NULL), customer_snapshot (JSONB — immutable customer/account billing name, address, email, language, tax ID per R-C-1), triggering_event_ref (the Kafka event ID that caused this invoice — used for idempotency), original_invoice_id (NULL except for credit/debit notes), issue_date, due_date (NULL for tax invoices), subtotal_amount, tax_amount_total, tax_summary (JSONB structured per R-X-3), total_amount, paid_amount, status, payment_status, period_start, period_end, and lifecycle timestamps.	Invoice record schema
R-BIL-02-E-1a	account_id is the billing and collection partition key for every invoice and pro forma. customer_id is stored only as denormalized identity/search context. One customer with two accounts must receive separate account-scoped invoice streams unless an explicit future account-consolidation product is approved.	Account anchor
R-BIL-02-E-2	A pro forma is stored in the separate pro_forma table. Same conceptual structure as invoice but: non-gap-free sequence, no due date (informational only), no payment_status, terminal states SUPERSEDED / CANCELLED / FULFILLED. Stored separately so legal-compliance queries on invoice table return zero pro forma rows.	Pro forma record schema
R-BIL-02-E-3	Line items are stored in shared invoice_line_item table. Each line: id, parent_type (INVOICE / PRO_FORMA), parent_id (FK to invoice.id or pro_forma.id), line_level (SUMMARY / DETAIL), parent_summary_line_id (NULL for SUMMARY, FK to SUMMARY line for DETAIL), wallet_type_code (FK to PLM-CFG-03 wallet type catalog; NULL for lines not tied to a wallet — rare), line_type (extensible — PACKAGE_CHARGE, EQUIPMENT_CHARGE, INSTALL_FEE, MACD_FEE, ETF, LATE_FEE, DISCOUNT, ADJUSTMENT_CREDIT, ADJUSTMENT_DEBIT, etc.), description, quantity, unit_price, subtotal, tax_breakdown (JSONB structured per R-W-X-2), tax_total, total, optional package_ref, equipment_ref, period_start, period_end, sort_order, metadata.	Line item schema
R-BIL-02-E-4	Line items are immutable. Corrections happen via adjustments (BIL-02-ADJ-01) which create new credit/debit note invoices, not by editing existing lines.	Line item immutability

ID	Rule	Triggered on
R-BIL-02-E-5	Invoice records are immutable except for status, payment_status, paid_amount, and lifecycle timestamps. Every change to these fields is recorded in the invoice_history change history table. The history records: who changed it, when, the old value, the new value, the reason code if applicable. History rows are never edited or deleted.	Invoice immutability + history

Lifecycle rules (Wananchi-aligned)

ID	Rule	Triggered on
R-BIL-02-L-1	Legal invoice statuses: GENERATED (just created) → PENDING (notification confirmed delivery to customer, awaiting payment) → PAID (full payment) OR OVERDUE (due date passed without full payment). OVERDUE → PAID on late payment. Any status can transition to CANCELLED (admin action). The 5-state model matches Wananchi workshop IN-05.	Lifecycle
R-BIL-02-L-2	Pro forma lifecycle: GENERATED → PRESENTED (notification delivered) → terminal states SUPERSEDED (next cycle's pro forma generated, replacing this), CANCELLED (subscription terminated or paused before next cycle), FULFILLED (customer paid for the corresponding cycle).	Pro forma lifecycle
R-BIL-02-L-3	Tax invoice has additional signing-flow states: PENDING_SIGNATURE → SIGNED → GENERATED (then standard lifecycle). On signing failure: SIGNATURE_FAILED. Owned by BIL-02-TAX-01 with full retry/recovery rules.	Tax invoice extended lifecycle
R-BIL-02-L-4	Status transitions are derived from events, never directly mutated by external callers. Core defines the transition rules; satellites trigger them via internal API. Triggers: notification module InvoicePresented → core transitions to PENDING; BIL-02-STA-01 overdue scanner → core transitions to OVERDUE; BIL-02-GEN-01 PaymentApplied consumer → core transitions to PAID; admin action → core transitions to CANCELLED.	Transition discipline
R-BIL-02-L-5	The Subscription's outstanding_balance master field is recomputed from BIL-02 events. Core emits InvoiceIssued, InvoicePaymentApplied, InvoiceCancelled (along with satellite events from BIL-02-ADJ-01 for adjustments). BIL-01 consumes these and writes the recomputed balance via SUB-LM-01. Pro forma never contributes to outstanding balance.	Outstanding balance
R-BIL-02-L-6	Cancelled invoices are retained with full audit (status, who, when, reason). Cancellation requires cancel_reason_code and optional cancel_batch_id (set during bulk reversal — see BIL-02-GEN-01).	Cancellation discipline

Numbering rules

ID	Rule	Triggered on
R-BIL-02-N-1	Each legal invoice (LEGAL_INVOICE + TAX_INVOICE + CREDIT_NOTE + DEBIT_NOTE) receives a legal_invoice_number immediately on creation, sequential per-(operator, fiscal_year, invoice_type_code), gap-free.	Legal invoice creation
R-BIL-02-N-2	V3 launch formats (configurable in invoice_type_operator_config.legal_number_format): KE: WIK-2026-NNNNNNNN for regular invoices, WIK-T-2026-NNNNNNNN for tax invoices, WIK-CN-2026-NNNNNNNN for credit notes, WIK-DN-2026-NNNNNNNN for debit notes. UG: WIU-*. TZ: WIT-*. SN: no legal invoice (pro forma only); pro forma YAS-PF-2026-NNNNNNNN.	Format examples

ID	Rule	Triggered on
R-BIL-02-N-3	Sequence allocation is atomic within the invoice insert transaction. SELECT FOR UPDATE on invoice_sequence row → increment → insert invoice. Sequence resets each fiscal year per fiscal_year_start_month. Implementation in InvoiceSequenceService (core).	Sequence allocation
R-BIL-02-N-4	Gap-free guarantee: if invoice creation fails after sequence allocation (rare — DB constraint, line item write failure), the skipped number is recorded in invoice_sequence_gap table for tax-authority reconciliation. InvoiceSequenceGap Kafka event fires. Operator admin console surfaces the gap queue for resolution. Never silently lost.	Gap audit
R-BIL-02-N-5	Pro forma uses its own sequence in pro_forma_sequence, per-(operator, fiscal_year). Non-gap-free — pro forma is informational, gaps are acceptable.	Pro forma numbering

Wallet grouping policy + multi-tax summary

ID	Rule	Triggered on
R-BIL-02-W-1	Charges produced at cycle close (and at one-off events) are distributed across one or more invoices according to the per-operator grouping policy , stored in invoice_grouping_config. The policy has a dimension indicating how grouping is decided: WALLET (group by wallet type), PACKAGE (group by package), SERVICE_CATEGORY (group by service category from PLM-CFG-04 Service Catalog), or SINGLE (no grouping — all charges on one invoice). The policy also has a list of groups ; each group specifies the keys (wallet codes, package refs, or category codes, depending on dimension) that belong together on one invoice. Number of invoices produced per cycle per customer = number of groups in the policy that have non-zero charges.	Grouping policy concept
R-BIL-02-W-2	The policy is set per (operator_code, invoice_type_code). Different invoice types can have different policies (and different dimensions) for the same operator — e.g., an operator might group cycle invoices by wallet but always produce a separate one-off invoice per individual package.	Per-type policy
R-BIL-02-W-3	Constraints on the policy (enforced by admin UI + DB constraint): every key that could appear in the operator's charge stream must be covered by exactly one group. For WALLET dimension: every wallet_type_code in PLM-CFG-03 Wallet Configuration for this operator must appear in exactly one group. For PACKAGE dimension: similar coverage of package refs (validated against PLM-CFG-01 Package Catalog). For SERVICE_CATEGORY: same against PLM-CFG-04. For SINGLE: no group keys needed; everything is in the one implicit group. No key may be in two groups (would double-bill); no key may be missing (would lose charges).	Coverage constraint
R-BIL-02-W-4	Common policy shapes — examples in each dimension: WALLET dimension : (a) all-grouped: one group containing all wallet types; (b) all-separate: N groups, one per wallet type; (c) hybrid like "2 wallets together + 1 separate" (the Wananchi KE case). PACKAGE dimension : per-package invoicing — useful for B2B/enterprise customers wanting separate invoice per package for accounting reconciliation. SERVICE_CATEGORY dimension : groups defined by service category catalog (e.g., "Connectivity" includes Internet packages, "Voice" includes Voice packages, "Equipment Rental" includes equipment). SINGLE dimension : one combined invoice covering everything regardless of wallet/package/category.	Policy shapes per dimension

ID	Rule	Triggered on
R-BIL-02-W-5	When an invoice covers a single key (one wallet, one package, etc.): the invoice's header carries that key in the appropriate column (wallet_type_code for WALLET dimension, package_ref for PACKAGE, service_category_code for SERVICE_CATEGORY). When an invoice covers multiple keys in a group: the invoice's header has NULL in the key column; line items carry their individual keys. The header column grouping_key_values JSONB always captures the array of keys this invoice covers, regardless of dimension — used for display and traceability.	Invoice header semantics
R-BIL-02-W-6	Generator behavior (BIL-02-GEN-01 Invoice Generation) at cycle close: look up the operator's grouping policy for the invoice type; for each group, filter BIL-01-returned charges by the group's keys (matching on the policy's dimension); generate one invoice per group with those line items. Each invoice has its own legal invoice number, due date, PDF, payment status, and dunning state. Idempotency tracks each separately via (triggering_event_ref, invoice_type_code, group_index).	Generator behavior
R-BIL-02-W-7	Changing the grouping policy on a live operator: takes effect for newly-generated invoices only. Existing open invoices remain in their original form (no retro-regrouping). Admin UI flags the transition: shows how many open invoices would have a different layout under the new policy. Validation: cannot change policy while any invoice for that (operator, invoice_type) is in OVERDUE status. Changing the dimension (e.g., WALLET → PACKAGE) is a major operation requiring Finance approval; admin UI requires explicit acknowledgment.	Policy migration
R-BIL-02-W-8	Trivial cases: single-wallet operators (e.g., Yas SN today) typically use SINGLE dimension. Operators with one package per customer per cycle don't need grouping at all and use SINGLE. The dimension and group count are operational choices, not technical constraints.	Trivial cases

ID	Rule	Triggered on
R-BIL-02-X-1	An invoice may carry multiple taxes that stack (e.g., KE: VAT 16% + Excise 15%; UG: VAT 18% + telecom levy; SN: VAT 18% + RUTEL levy). The exact tax rules — which taxes apply, cascade order, base computation, exemptions, B2B reverse-charge — are owned entirely by PLM-CFG-02 Tax Configuration. BIL-02 calls TaxService.computeTax(lineItem, customerLocation, packageRef) per line item and stores the returned breakdown.	Tax computation delegation
R-BIL-02-X-2	Per-line tax_breakdown JSONB structure (returned by TaxService and stored on invoice_line_item.tax_breakdown): an array of tax-component records. Each record: {taxCode, taxName, ratePercent, baseAmount, taxAmount, cascadeLevel}. Example for a KES 4000 telecom line item in Kenya: [{taxCode: "EXCISE", taxName: "Excise Duty", ratePercent: 15, baseAmount: 4000, taxAmount: 600, cascadeLevel: 1}, {taxCode: "VAT", taxName: "VAT", ratePercent: 16, baseAmount: 4600, taxAmount: 736, cascadeLevel: 2}]. The line's tax_total = 600 + 736 = 1336.	Per-line tax breakdown

ID	Rule	Triggered on
R-BIL-02-X-3	Invoice-level tax_summary JSONB is computed at generation time by aggregating per-line tax_breakdown across all line items. Structure: an array of per-tax-component totals across the invoice. Example: [{taxCode: "EXCISE", taxName: "Excise Duty", totalBaseAmount: 4000, totalTaxAmount: 600}, {taxCode: "VAT", taxName: "VAT", totalBaseAmount: 4600, totalTaxAmount: 736}]. The invoice's tax_amount_total is the sum (= 1336). This structure supports any number of stacked taxes — V3 launch operators need 1-3, future deployments may need more.	Invoice-level tax summary
R-BIL-02-X-4	The tax_summary array is what gets reported to tax authorities for filing (one summary per tax code per invoice). It's also what gets rendered on the customer invoice PDF (one line per tax code in the tax-breakdown section). The structured JSONB is stable across operator-specific tax rules — only PLM-CFG-02 Tax Configuration needs to know which taxes apply where.	Reporting use of tax_summary
R-BIL-02-X-5	A flat tax_amount scalar column is intentionally NOT used. Reducing to a single number loses the breakdown needed for compliance reporting and customer PDF rendering. The denormalized tax_amount_total is convenience-only (matches the sum of tax_summary entries — kept for performance on listing queries).	No flat scalar

Customer snapshot (immutable customer state on the invoice)

ID	Rule	Triggered on
R-BIL-02-C-1	At invoice generation, the generator captures a snapshot of the customer and account billing details and stores it on the invoice in the customer_snapshot JSONB column. The snapshot is read from the CRM/customer service at the moment of issuance and frozen onto the invoice row. Snapshot fields: full_name, billing_account_label, billing_address (line1, line2, city, region, postal_code, country_code), email, phone, language_preference, tax_identifier (type + value, e.g., KRA_PIN for Kenya), customer_type (INDIVIDUAL / BUSINESS / GOVERNMENT), business_registration_number (for BUSINESS customers), snapshot_taken_at (timestamp).	Snapshot at issuance
R-BIL-02-C-2	The snapshot is immutable for that invoice. If the customer changes their billing address 6 months after issuance, the historical invoice still carries the old address — reflecting customer state at the time of issuance. Required for legal compliance: a 2026 tax invoice reproduced in 2030 must show the customer info as of 2026, not as of 2030. Tax authorities verify customer details against their records of who was billed when.	Immutability of snapshot
R-BIL-02-C-3	Different documents may capture snapshots at different moments and therefore carry different customer state. A regular cycle invoice snapshots customer state at cycle close. The corresponding tax invoice (generated later on payment) snapshots customer state at payment time — may differ if the customer updated details between cycle and payment. Credit notes snapshot customer state at credit-note issuance. Each document is self-contained for its own moment.	Per-document snapshot

ID	Rule	Triggered on
R-BIL-02-C-4	The snapshot makes the invoice a self-contained document for rendering : the notification module receives the invoice + line items + customer_snapshot in the InvoiceIssued event payload and has everything needed to render the PDF without further callbacks to CRM. This matters for (a) PDF re-rendering years later when CRM may have evolved, (b) regulatory robustness (PDF reproducible from invoice data alone), and (c) reduced cross-module coupling at render time.	Self-contained rendering
R-BIL-02-C-5	What's NOT in the snapshot (stays in the notification module): channel preferences (email, SMS, portal, push), template version selection, branding choices, delivery-time preferences (quiet hours), bounce/delivery state. These can change without invalidating the historical invoice; they're delivery metadata, not invoice content.	Snapshot scope boundary
R-BIL-02-C-6	For B2B customers with separate billing-contact vs technical-contact, the snapshot uses the billing contact details (per CRM's billing_contact field). Technical contact info is irrelevant for invoice rendering.	Billing-contact for B2B
R-BIL-02-C-7	If CRM is unavailable at generation time, the generator retries (per generation framework). If CRM remains unavailable beyond retry limit, the invoice generation is queued in generation_failure_queue (BIL-02-GEN-01 Invoice Generation) for resolution. The invoice is NOT generated with a partial snapshot or with NULL customer fields.	CRM unavailability

Subscription state interactions

ID	Rule	Triggered on
R-BIL-02-S-1	BIL-02 does not generate any document for Subscriptions in PENDING_* (mid-workflow) status. Trigger events check Subscription status via SUB-LM-01 snapshot, skip if mid-workflow, schedule retry for now() + 1 hour. Long-running PENDING_* (>4 hours) is flagged via InvoiceSkippedDueToPendingTooLong Kafka event for operator review.	PENDING gating
R-BIL-02-S-2	BIL-02 reads Subscription master fields (account_id, customer_id, billing_mode, customer_location, currency, current_cycle_start, current_cycle_end) during generation. For PREPAID subscribers, writes current_cycle_proforma_id back via SUB-LM-01 put-master-fields.	Master field interaction
R-BIL-02-S-3	On Subscription termination (SUB-WF-TERMINATE-01), BIL-02 receives no new invoice trigger. Final settlement (ETF, equipment write-off, final cycle proration) flows as line items on the final regular invoice (POSTPAID, issued at the cycle close in which termination occurs) or as final-cycle adjustments handled by BIL-05 Wallet and Topup (PREPAID — whether the operator uses a wallet or direct-pay model). The system supports adding a separate TERMINATION invoice type later via the extensible catalog if a future deployment requires it.	Termination
R-BIL-02-S-4	On Subscription non-payment suspension (BIL-04): BIL-03 stops cycle close → no new cycle invoice generated; pro forma scanner skips SUSPENDED Subscriptions. Existing OVERDUE invoices remain OVERDUE; BIL-04 owns dunning state. On recovery: cycle close resumes; pro forma resumes.	Suspension handling

3. Module owner and impacted modules

Role	Module	What it does
Owner	New billing-invoicing module on cluster C	Owms: invoice + pro forma entities, line items, type catalog, sequence allocation, per-operator config, multi-tax summary, wallet isolation strategy, scanner framework, REST + Kafka contract. Satellite DDs add their generators, scanners, and admin endpoints to the same module.
Upstream events consumed	notification module (InvoicePresented ack — for GENERATED → PENDING transition)	Source events drive lifecycle transitions.
Callees via REST	SUB-LM-01 (put-master-fields for current_cycle_pro_forma_id)	Synchronous REST.
Downstream consumers via Kafka	BIL-01 (consumes InvoiceIssued, InvoicePaymentApplied, InvoiceCancelled for outstanding balance), notification module (consumes events for customer comms), analytics, CVM, audit	Downstream consumers.
Frontend	Admin Invoice Console (read views, type catalog admin, per-operator config admin, sequence-gap queue); Customer Portal (read views)	Admin + customer self-service. Satellite DDs add their admin actions and customer screens.
Engines	No Drools in core. Tax rules in PLM-CFG-02 Tax Configuration.	—

4. Foundation references

DB → FOUNDATION_DB.md · Kafka → FOUNDATION_KAFKA.md · Auth → FOUNDATION_AUTH.md. Framework: SUB-WF-FRAMEWORK-01.

Technical design

Module placement

Layer	Where
Frontend	Admin Invoice Console + Customer Portal invoice/document module. Satellites add their UI screens.
Backend	New billing-invoicing module on cluster C. Satellite DDs co-locate in the same module.
Rules	None in core. PLM-CFG-02 owns tax rules.
Events consumed (core only)	1 event (InvoicePresented from notification for PENDING transition). Satellites add their own consumed events.
Events emitted (core only)	4 events: InvoiceIssued, InvoicePaymentApplied, InvoiceCancelled, InvoiceSequenceGap, InvoiceSkippedDueToPendingTooLong. Satellites emit their own.

Data model

invoice_type_definition — extensibility catalog

Column	Type	Notes
type_code	TEXT PK	CYCLE_POSTPAID, ONE_OFF_INVOICE, PRO_FORMA_CYCLE, TAX_INVOICE, CREDIT_NOTE, DEBIT_NOTE. Future deployments add more.
display_name	TEXT	
legal_nature	TEXT	LEGAL_INVOICE / PRO_FORMA / TAX_INVOICE / ADJUSTMENT_NOTE
generator_class_ref	TEXT	Class implementing InvoiceGenerator interface (owned by BIL-02-GEN-01, BIL-02-TAX-01, BIL-02-ADJ-01)
trigger_type	TEXT	EVENT_DRIVEN / SCHEDULED_SCANNER / ADMIN_ACTION
requires_signing	BOOLEAN	True for tax invoices (delegated to BIL-02-TAX-01)
legal_sequence_required	BOOLEAN	True for legal invoices (gap-free)

Column	Type	Notes
customer_deliverable	BOOLEAN	Does customer receive
active	BOOLEAN	Soft-disable
created_at, updated_at	TIMESTAMPTZ	

Change history table (invoice_type_definition_history) tracks every change.

Sample data — what the catalog looks like in V3:

type_code	display_name	legal_nature	generator_class_ref	trigger_type	requires_signing	legal_seq_req'd	active
CYCLE_POSTPAID	Regular Cycle Invoice	LEGAL_INVOICE	CyclePostpaid InvoiceGenerator	EVENT_DRIVEN	false	true	true
ONE_OFF_INVOICE	One-off (MACD / Setup)	LEGAL_INVOICE	OneOffInvoiceGenerator	EVENT_DRIVEN	false	true	true
PRO_FORMA_CYCLE	Pro Forma — Next Cycle	PRO_FORMA	ProFormaCycleGenerator	SCHEDULED_SCANNER	false	false	true
TAX_INVOICE	Signed Tax Invoice	TAX_INVOICE	TaxInvoiceGenerator	EVENT_DRIVEN	true	true	true
CREDIT_NOTE	Credit Note	ADJUSTMENT_NOTE	CreditNoteGenerator	ADMIN_ACTION	false	true	true
DEBIT_NOTE	Debit Note	ADJUSTMENT_NOTE	DebitNoteGenerator	ADMIN_ACTION	false	true	true

How to read this: type_code is the unique key referenced everywhere. legal_nature distinguishes real legal documents from informational pro forma. generator_class_ref names the class that produces this type of document. requires_signing is true only for tax invoices. legal_seq_req'd (gap-free numbering) is true for everything customer-facing except pro forma.

invoice_type_operator_config — per-operator activation

Column	Type	Notes
id	TEXT PK	ULID
operator_code	TEXT NOT NULL	
invoice_type_code	TEXT NOT NULL	FK
enabled	BOOLEAN	Generate this type for this operator
signing_service_endpoint	TEXT NULL	Tax GW URL (used by BIL-02-TAX-01)
signing_service_implementation_ref	TEXT NULL	Per-operator signer impl
due_date_grace_days	INTEGER NULL	Default per type
pre_due_reminder_window_days	INTEGER NULL	Used by BIL-02-STA-01
pre_cycle_pro_forma_window_days	INTEGER NULL	Used by BIL-02-GEN-01 pro forma scanner
legal_number_format	TEXT NULL	Operator format string
fiscal_year_start_month	INTEGER DEFAULT 1	
digits_in_sequence	INTEGER DEFAULT 8	
cold_storage_age_days	INTEGER DEFAULT 365	Used by BIL-02-READ-01 Invoice Read API (if cold-storage tiering is implemented downstream of caching)
created_at, updated_at	TIMESTAMPTZ	

Additional per-operator config fields are added by satellite DDs (e.g., late_fee_* by BIL-02-STA-01, adjustment_csr_threshold by BIL-02-ADJ-01). All flow through this same table.

(operator_code, invoice_type_code) UNIQUE. Change history table (invoice_type_operator_config_history) tracks every change to per-operator config.

Sample data — per-operator activation matrix at V3 launch:

operator_code	invoice_type_code	enabled	due_grace_days	pre_due_reminder_days	pre_cycle_proforma_window_days	legal_number_format	signing_endpoint
WANANCHI_KE	CYCLE_POSTPAID	true	14	3	—	WIK-{YYYY}-{NNNNNNNN}	—
WANANCHI_KE	ONE_OFF_INVOICE	true	14	3	—	WIK-OF-{YYYY}-{NNNNNNNN}	—
WANANCHI_KE	PRO_FORMA_CYCLE	true	—	—	5	WIK-PF-{YYYY}-{NNNNNNNN}	—
WANANCHI_KE	TAX_INVOICE	true	—	—	—	WIK-T-{YYYY}-{NNNNNNNN}	https://autosend.wananchi.co.ke/kra-etims/v1/sign
WANANCHI_KE	CREDIT_NOTE	true	—	—	—	WIK-CN-{YYYY}-{NNNNNNNN}	—
WANANCHI_UG	CYCLE_POSTPAID	true	14	3	—	WIU-{YYYY}-{NNNNNNNN}	—
WANANCHI_UG	TAX_INVOICE	false	—	—	—	—	—
WANANCHI_TZ	CYCLE_POSTPAID	true	14	3	—	WIT-{YYYY}-{NNNNNNNN}	—
WANANCHI_TZ	TAX_INVOICE	false	—	—	—	—	—
YAS_SN	PRO_FORMA_CYCLE	true	—	—	5	YAS-PF-{YYYY}-{NNNNNNNN}	—
YAS_SN	TAX_INVOICE	false	—	—	—	—	—

How to read this:

- Each row = one (operator, invoice_type) combination with its specific settings.
- Yas SN only has PRO_FORMA_CYCLE enabled — they're a PREPAID-only operator (customers pay before cycle, no POSTPAID customers) with no tax invoice requirement.
- Wananchi KE has TAX_INVOICE enabled with the Autosend Gateway endpoint pointing to their KRA Etims proxy.
- Wananchi UG/TZ have TAX_INVOICE disabled in this sample; operator toggles it per their deployment.
- The legal_number_format template uses {YYYY} for the fiscal year and {NNNNNNNN} for the zero-padded sequence number. The system substitutes these at allocation time.
- Grouping policy for each operator is configured separately in invoice_grouping_config (next table).

invoice_grouping_config — per-operator grouping policy (dimension + groups)

Per R-W-1 through R-W-8. Defines how charges are distributed across invoices for each (operator, invoice_type) combination. The policy has a dimension (WALLET / PACKAGE / SERVICE_CATEGORY / SINGLE) and one or more groups; each group is one row.

Column	Type	Notes
id	TEXT PK	ULID
operator_code	TEXT NOT NULL	
invoice_type_code	TEXT NOT NULL	Different types can have different policies for the same operator
grouping_dimension	TEXT NOT NULL	WALLET / PACKAGE / SERVICE_CATEGORY / SINGLE. All groups under the same (operator, invoice_type) must share the same dimension (you can't mix-and-match dimensions within one policy).
group_index	INTEGER NOT NULL	1, 2, 3 — display order
group_name	TEXT NULL	Operator-friendly label (e.g., "Internet & TV bundle", "Package A", "Voice line") — used as the customer-facing label on the invoice PDF
group_key_values	TEXT[] NOT NULL	Array of keys this group covers. Content depends on grouping_dimension: wallet_type_codes for WALLET, package_refs for PACKAGE, service_category_codes for SERVICE_CATEGORY, empty array for SINGLE.
created_at, updated_at	TIMESTAMPTZ	

Indexes: (operator_code, invoice_type_code, group_index) UNIQUE; (operator_code, invoice_type_code) for lookup at generation time. Constraint (admin UI + DB check): for a given (operator, invoice_type), all groups share the same grouping_dimension; the union of all group_key_values covers every applicable key in the operator's catalog (PLM-CFG-03 Wallet Configuration for WALLET dimension, PLM-CFG-01 Package Catalog for PACKAGE, PLM-CFG-04 Service Catalog for SERVICE_CATEGORY); the intersection across groups is empty. SINGLE dimension has exactly one group with empty group_key_values. Change history table (invoice_grouping_config_history) tracks policy edits — every change requires Finance approval.

Sample data — policies for V3 launch operators:

*Wananchi KE with 3 wallet types (WALLET_INTERNET, WALLET_TV, WALLET_VOICE) using **WALLET dimension** with hybrid grouping: Internet+TV combined, Voice separate.*

id	operator_code	invoice_type_code	grouping_dimension	group_index	group_name	group_key_values
gc_01...A1	WANANCHI_KE	CYCLE_POSTPAID	WALLET	1	Internet & TV	[WALLET_INTERNET, WALLET_TV]
gc_01...A2	WANANCHI_KE	CYCLE_POSTPAID	WALLET	2	Voice	[WALLET_VOICE]

Result: at cycle close, customer with all 3 wallets receives **2 invoices** — one for Internet+TV, one for Voice. Each independent.

*Wananchi UG with same 3 wallet types but **all grouped onto one invoice** — using WALLET dimension, single group:*

id	operator_code	invoice_type_code	grouping_dimension	group_index	group_name	group_key_values
gc_01...B1	WANANCHI_UG	CYCLE_POSTPAID	WALLET	1	All services	[WALLET_INTERNET, WALLET_TV, WALLET_VOICE]

Result: one combined invoice covering all 3 wallets.

*Yas SN single-wallet operator using **SINGLE dimension** (simplest case — no grouping logic needed):*

id	operator_code	invoice_type_code	grouping_dimension	group_index	group_name	group_key_values
gc_01...C1	YAS_SN	PRO_FORMA_CYCLE	SINGLE	1	Service	[]

Result: one pro forma per cycle covering everything. SINGLE dimension means "ignore wallets/packages/categories — just put it all on one invoice".

*Hypothetical enterprise operator wanting **per-package invoicing** (B2B accounting use case) — using PACKAGE dimension:*

id	operator_code	invoice_type_code	grouping_dimension	group_index	group_name	group_key_values
gc_01...D1	ENTERPRISE_KE	CYCLE_POSTPAID	PACKAGE	1	Package — Business Pro	[PKG_BUSINESS_PRO]
gc_01...D2	ENTERPRISE_KE	CYCLE_POSTPAID	PACKAGE	2	Package — Voice Enterprise	[PKG_VOICE_ENTERPRISE]
gc_01...D3	ENTERPRISE_KE	CYCLE_POSTPAID	PACKAGE	3	Package — Equipment Lease	[PKG_EQUIPMENT_LEASE]

Result: customer with all 3 packages receives **3 separate invoices**, one per package. Useful for enterprise accounting reconciliation where each PO/cost-center references a specific package.

Hypothetical operator grouping by service category — using SERVICE_CATEGORY dimension:

id	operator_code	invoice_type_code	grouping_dimension	group_index	group_name	group_key_values
gc_01...E1	FUTURE_OP	CYCLE_POSTPAID	SERVICE_CATEGORY	1	Connectivity	[CAT_INTERNET, CAT_TV]
gc_01...E2	FUTURE_OP	CYCLE_POSTPAID	SERVICE_CATEGORY	2	Voice	[CAT_VOICE]
gc_01...E3	FUTURE_OP	CYCLE_POSTPAID	SERVICE_CATEGORY	3	Equipment	[CAT_EQUIPMENT_RENTAL]

Result: 3 invoices per cycle, grouped by service category. Internet and TV packages share an invoice because they're in the same Connectivity category.

How to read this:

- The **dimension** decides what grouping axis applies: wallet, package, service category, or none (SINGLE).
- The system generates **one invoice per group** that has non-zero charges in the cycle.
- group_name appears on the invoice PDF as the section header (or the only-section name on a single-group invoice).
- Different operators can use different dimensions. Wananchi groups by wallet (operationally meaningful for them); a B2B operator might group by package (accounting-meaningful); some operators just want one bill for everything (SINGLE).
- Operators can change the policy via admin UI; existing open invoices stay in their original form. Changing the dimension itself requires Finance approval and triggers an admin warning showing impact.

invoice — legal invoices

Column	Type	Notes
id	TEXT PK	ULID — system-generated unique invoice identifier
invoice_type_code	TEXT NOT NULL	FK to type catalog (e.g., CYCLE_POSTPAID)
legal_invoice_number	TEXT UNIQUE NOT NULL	Gap-free per (operator, fiscal_year, type). The customer-visible number on the PDF.
subscription_id	TEXT NOT NULL	The Subscription this invoice belongs to
customer_id	TEXT NOT NULL	Denormalized from Subscription for query convenience
operator_code	TEXT NOT NULL	WANANCHI_KE, WANANCHI_UG, WANANCHI_TZ, YAS_SN, etc.
currency	TEXT NOT NULL	ISO 4217 (KES, UGX, TZS, XOF, etc.)
billing_mode	TEXT NOT NULL	POSTPAID or PREPAID. Captured at issuance from the Subscription master snapshot. Immutable — if the customer converts to a different billing mode later, this invoice still reflects the mode at issuance. Useful for reporting ("show all POSTPAID invoices for Q3") and audit.
grouping_config_ref	TEXT NOT NULL	FK to invoice_grouping_config.id — identifies which group of the policy this invoice represents.
grouping_dimension	TEXT NOT NULL	Denormalized from config row at issuance: WALLET / PACKAGE / SERVICE_CATEGORY / SINGLE. Recorded on the invoice so future PDF re-renders know which dimension was in force at issuance (operator may have changed policy since).
grouping_key_values	JSONB NOT NULL	Array of the keys this invoice covers (e.g., ["WALLET_INTERNET", "WALLET_TV"] or ["PKG_BUSINESS_PRO"]). Empty array for SINGLE dimension.
wallet_type_code	TEXT NULL	Set ONLY when grouping_dimension=WALLET AND invoice covers exactly one wallet; otherwise NULL. Convenience column for query/index.
package_ref	TEXT NULL	Set ONLY when grouping_dimension=PACKAGE AND invoice covers exactly one package; otherwise NULL. Convenience.
service_category_code	TEXT NULL	Set ONLY when grouping_dimension=SERVICE_CATEGORY AND invoice covers exactly one category; otherwise NULL. Convenience.
customer_snapshot	JSONB NOT NULL	Immutable customer state at issuance per R-C-1. Contains: full_name, billing_address (line1, line2, city, region, postal_code, country_code), email, phone, language_preference, tax_identifier (type + value), customer_type, optional business_registration_number, snapshot_taken_at.
triggering_event_ref	TEXT NOT NULL	What caused this invoice to be generated. For cycle invoices: the cycle-close event ID from BIL-03 Cycle Close (e.g., cycle_close_01HZQX...). For one-off invoices: the OneOffChargeRequested event ID. For tax invoices: the PaymentApplied event ID. Used for idempotency.

Column	Type	Notes
original_invoice_id	TEXT NULL	For credit/debit notes — set by BIL-02-ADJ-01 Invoice Adjustments to point to the invoice being adjusted. NULL for normal cycle/one-off/tax invoices.
issue_date	TIMESTAMPTZ NOT NULL	When the invoice was created. Used as the partition key — invoices are stored in monthly partitions by issue_date.
due_date	TIMESTAMPTZ NULL	When the customer must pay. Computed as issue_date + grace_days. NULL for TAX_INVOICE (it's already paid — proof-of-payment doc).
subtotal_amount	NUMERIC(15,2)	Sum of all line subtotals before tax
tax_amount_total	NUMERIC(15,2)	Sum of all tax components across the invoice — denormalized from tax_summary for query convenience
tax_summary	JSONB	Structured per-tax-component totals across the invoice (R-X-3). Example: [{taxCode:"EXCISE", totalBase:4000,totalTax:600},{taxCode:"VAT",totalBase:4600,totalTax:736}]
total_amount	NUMERIC(15,2)	subtotal + tax_amount_total (negative for credit notes)
paid_amount	NUMERIC(15,2) DEFAULT 0	Running total of payments received against this invoice
status	TEXT NOT NULL	GENERATED / PENDING / PAID / OVERDUE / CANCELLED per R-L-1
payment_status	TEXT DEFAULT 'UNPAID'	UNPAID / PARTIALLY_PAID / PAID
presented_at, paid_at, overdue_at, cancelled_at	TIMESTAMPTZ NULL	Timestamps for each lifecycle state — set when transition happens
cancel_reason_code	TEXT NULL	If status = CANCELLED: why (e.g., BULK_REVERSAL, OPERATOR_VOIDED, DUPLICATE_DETECTED)
cancel_batch_id	TEXT NULL	If cancelled as part of a bulk reversal: the batch ID — links to bulk_reversal_batch in BIL-02-GEN-01 Invoice Generation
period_start, period_end	TIMESTAMPTZ NULL	For cycle invoices: the start and end of the cycle being billed. NULL for one-off invoices not tied to a period.
metadata	JSONB NULL	Free-form per-operator data (e.g., PO number for enterprise customers, additional regulatory fields). Operator-specific schema. NOT for customer info — that's in customer_snapshot.

Indexes: (legal_invoice_number) UNIQUE; (subscription_id, issue_date DESC); (triggering_event_ref, invoice_type_code, grouping_config_ref) UNIQUE — idempotency includes the specific group; (status, due_date) partial WHERE status IN ('GENERATED','PENDING'); (operator_code, issue_date); (invoice_type_code, status) partial WHERE invoice_type_code='TAX_INVOICE'; (grouping_dimension) for analytics queries.

Monthly partitions by issue_date. P10Y retention. Change history table (invoice_history) tracks every change to status, payment_status, paid_amount, and lifecycle timestamps with who/when/old-value/new-value. The customer_snapshot is **never** mutated after issuance — no history row needed for that column.

Sample data — three realistic invoices:

Example 1 — Wananchi KE POSTPAID cycle invoice covering the "Internet & TV" group (Wananchi KE policy is WALLET dimension with 2 groups; this is invoice #1 of 2 generated):

Column	Value
id	inv_01HZQX5K3M9P0V8YQ6
invoice_type_code	CYCLE_POSTPAID
legal_invoice_number	WIK-2026-00012345
subscription_id	sub_01HVR3T2W5K8
customer_id	cust_01HVR3T1A9F2

Column	Value
operator_code	WANANCHI_KE
currency	KES
billing_mode	POSTPAID
grouping_config_ref	gc_01...A1 (the "Internet & TV" group from Wananchi KE's CYCLE_POSTPAID policy)
grouping_dimension	WALLET
grouping_key_values	["WALLET_INTERNET", "WALLET_TV"]
wallet_type_code	NULL (multiple wallets in this group → header has no single code; line items carry their individual wallet types)
package_ref	NULL
service_category_code	NULL
customer_snapshot	{ "full_name": "John Mwangi Kamau", "billing_address": { "line1": "Plot 45, Ngong Road", "line2": "Lavington", "city": "Nairobi", "region": "Nairobi County", "postal_code": "00100", "country_code": "KE"}, "email": "john.kamau@example.co.ke", "phone": "+254712345678", "language_preference": "EN", "tax_identifier": { "type": "KRA_PIN", "value": "A001234567Z"}, "customer_type": "INDIVIDUAL", "snapshot_taken_at": "2026-10-01T00:01:30Z" }
triggering_event_ref	cycle_close_01HXP8X4M3K7 (the BIL-03 Cycle Close event ID)
issue_date	2026-10-01 00:01:32+00
due_date	2026-10-15 23:59:59+00
subtotal_amount	5,200.00 (Internet + TV portion of the cycle — Voice is on a separate invoice per the hybrid policy)
tax_amount_total	1,202.50
tax_summary	[{ "taxCode": "EXCISE", "totalBaseAmount": 5200.00, "totalTaxAmount": 780.00 }, { "taxCode": "VAT", "totalBaseAmount": 5980.00, "totalTaxAmount": 422.50 }]
total_amount	6,402.50
paid_amount	0.00
status	GENERATED
payment_status	UNPAID
period_start	2026-09-01 00:00:00+00
period_end	2026-09-30 23:59:59+00
metadata	{ "po_number": null, "customer_ref": "WCH-12345" }

A **second invoice** is generated in the same cycle for the same customer covering Voice — separate row in the invoice table with grouping_config_ref = gc_01...A2, grouping_key_values = ["WALLET_VOICE"], wallet_type_code = WALLET_VOICE (single-wallet group → header carries the code), separate legal number WIK-2026-00012346. Same customer_snapshot (same customer at same moment).

Example 2 — Yas SN pro forma stored in pro_forma table (shown here for comparison; pro_forma sample below):

This is a pro forma, so it lives in the pro_forma table not the invoice table. See pro forma section below.

Example 3 — Wananchi KE Tax Invoice produced after the customer pays the invoice above (partial payment of 3,500 KES):

Column	Value
id	inv_01HZQX8N2R4M5T7W3F
invoice_type_code	TAX_INVOICE
legal_invoice_number	WIK-T-2026-00008721
subscription_id	sub_01HVR3T2W5K8
customer_id	cust_01HVR3T1A9F2
operator_code	WANANCHI_KE
currency	KES
billing_mode	POSTPAID (same as the original cycle invoice it was triggered by)

Column	Value
grouping_config_ref	gc_01...A1 (matches the original invoice's group)
grouping_dimension	WALLET
grouping_key_values	["WALLET_INTERNET", "WALLET_TV"] (same as original)
wallet_type_code	NULL
customer_snapshot	{"full_name": "John Mwangi Kamau", ..., "snapshot_taken_at": "2026-10-10T14:32:15Z"} (snapshot taken at payment moment — may differ slightly from the cycle invoice's snapshot if customer details changed)
triggering_event_ref	payment_01HZQX7M8B6C9D2 (the BIL-01 Charging Engine PaymentApplied event ID)
original_invoice_id	inv_01HZQX5K3M9P0V8YQ6 (links back to the cycle invoice in Example 1)
issue_date	2026-10-10 14:32:18+00
due_date	NULL (tax invoice has no due date — already paid)
subtotal_amount	2,841.66 (paid portion: 3,500/6,402.50 ≈ 54.66% of cycle invoice)
tax_amount_total	658.34
tax_summary	[{"taxCode": "EXCISE", "taxName": "Excise Duty", "totalBaseAmount": 2841.66, "totalTaxAmount": 426.25}, {"taxCode": "VAT", "taxName": "VAT", "totalBaseAmount": 3267.91, "totalTaxAmount": 232.09}] (proportional to the paid portion — recomputed via PLM-CFG-02 Tax Configuration TaxService, not just scaled from the original)
total_amount	3,500.00
paid_amount	3,500.00
status	SIGNED (was PENDING_SIGNATURE until Tax GW returned signature)
payment_status	PAID
paid_at	2026-10-10 14:30:00+00
metadata	{"signed_invoice_number": "KRA-ETIMS-2026-XQ7-49281", "tax_signature_data": {"qr": "...base64...", "tin": "P051234567X", "sigVersion": "v2"}, "signed_at": "2026-10-10T14:32:15Z", "original_payment_ref": "payment_01HZQX7M8B6C9D2"}

How to read this:

- triggering_event_ref always tells you WHAT caused the invoice to exist (cycle close event, payment event, etc.).
- tax_summary is the structured per-tax breakdown the system uses for tax-authority filing AND for rendering the customer PDF's tax section.
- customer_snapshot is the immutable customer state at issuance — used by the notification module to render the PDF without needing to call CRM.
- The tax invoice's metadata carries signing-specific fields (KRA-issued reference, QR code data) — these are BIL-02-TAX-01 Tax Invoice & Gateway's domain stored on the core invoice row.
- Notice how the tax invoice's amounts are **proportional** to the payment received (not the full original invoice) — partial payment of 3,500 / 6,402.50 ≈ 54.66% produces a tax invoice for that portion.

pro_forma — informational documents

Same conceptual structure as invoice, separate table. Columns: id, invoice_type_code, pro_forma_number (non-gap-free), subscription_id, customer_id, operator_code, currency, grouping_config_ref (FK to invoice_grouping_config.id), grouping_dimension, grouping_key_values (JSONB), wallet_type_code / package_ref / service_category_code (convenience columns, populated when applicable per same rules as invoice), customer_snapshot (JSONB, same structure as invoice), billing_mode (always PREPAID — pro forma is the pay-before-cycle document; POSTPAID customers don't get pro forma, they get a regular cycle invoice after the cycle), idempotency_cycle_key (synthetic key for scanner idempotency — NOT a reference to a real event, since pro forma is generated by a scheduled scanner, not from a Kafka event; format: cycle_{YYYY}_{MM}_sub_{subscription_id}), issue_date (partition key), cycle_start, cycle_end, subtotal_amount, tax_amount_total, tax_summary (same JSONB structure as invoice), total_amount, status (per R-L-2), presented_at, superseded_at, superseded_by_id, cancelled_at, fulfilled_at, metadata. Indexes: (subscription_id, cycle_end DESC); (subscription_id, idempotency_cycle_key, grouping_config_ref) UNIQUE; (status, cycle_start). Monthly partitions. P10Y retention. Change history table (pro_forma_history) tracks status changes.

Sample data — pro forma for a PREPAID Wananchi customer:

Column	Value
id	pf_01HZP4K2M5N8Q7R3T6
invoice_type_code	PRO_FORMA_CYCLE
pro_forma_number	WIK-PF-2026-00045892
subscription_id	sub_01HVR8M4N2P6
customer_id	cust_01HVR8M3K1L4
operator_code	WANANCHI_KE
currency	KES
grouping_config_ref	gc_01...A1 (Internet & TV group)
grouping_dimension	WALLET
grouping_key_values	["WALLET_INTERNET", "WALLET_TV"]
wallet_type_code	NULL
customer_snapshot	{"full_name": "Grace Wanjiku Ndegwa", "billing_address": {"line1": "House 12, Karen Crescent", "city": "Nairobi", "country_code": "KE"}, "email": "grace.ndegwa@example.co.ke", "phone": "+254723456789", "language_preference": "EN", "tax_identifier": {"type": "KRA_PIN", "value": "A009876543M"}, "customer_type": "INDIVIDUAL", "snapshot_taken_at": "2026-09-26T03:00:10Z"}
billing_mode	PREPAID
idempotency_cycle_key	cycle_2026_10_sub_01HVR8M4N2P6
issue_date	2026-09-26 03:00:12+00 (5 days before cycle end)
cycle_start	2026-10-01 00:00:00+00 (the upcoming cycle)
cycle_end	2026-10-31 23:59:59+00
subtotal_amount	4,000.00
tax_amount_total	925.00
tax_summary	[{"taxCode": "EXCISE", "totalBaseAmount": 4000.00, "totalTaxAmount": 600.00}, {"taxCode": "VAT", "totalBaseAmount": 4600.00, "totalTaxAmount": 325.00}]
total_amount	4,925.00
status	PRESENTED (notification delivered to customer)
presented_at	2026-09-26 03:04:55+00
metadata	{"customer_wallet_balance_at_issue": 3200.00, "top_up_recommended": 1725.00}

How to read this:

- Pro forma number is WIK-PF- . . . not WIK- . . . — separate sequence space from real invoices.
- cycle_start and cycle_end describe the **upcoming** cycle that will be billed, not the past one — this is a forward-looking notice.
- customer_snapshot is captured at the moment the pro forma is generated. The notification module uses this directly to render the SMS or email to the customer in their preferred language.
- metadata carries operator-useful context: customer's wallet balance at the moment of issuance and a recommended top-up. Notification embeds this in the message ("Your wallet has 3,200 KES — top up at least 1,725 KES before Oct 1.").
- When the customer pays the corresponding cycle, this pro forma's status moves to FULFILLED. If the customer doesn't pay and a new pro forma is generated next month, this one moves to SUPERSEDED with superseded_by_id pointing to the new pro forma.

invoice_line_item — shared line items for invoice + pro forma

Two-level structure per R-E-3.

Column	Type	Notes
id	TEXT PK	ULID
parent_type	TEXT NOT NULL	INVOICE / PRO_FORMA
parent_id	TEXT NOT NULL	FK to invoice.id or pro_forma.id

Column	Type	Notes
line_level	TEXT NOT NULL	SUMMARY / DETAIL
parent_summary_line_id	TEXT NULL	For DETAIL lines, FK to SUMMARY line
wallet_type_code	TEXT NULL	FK to PLM-CFG-03 wallet_type_definition. Set when line is tied to a specific wallet (most lines). NULL for cross-wallet items (rare).
line_type	TEXT NOT NULL	Extensible (PACKAGE_CHARGE, EQUIPMENT_CHARGE, etc.)
description	TEXT NOT NULL	
quantity	NUMERIC(15,4) DEFAULT 1	
unit_price	NUMERIC(15,2) NOT NULL	
subtotal	NUMERIC(15,2) NOT NULL	
tax_breakdown	JSONB NOT NULL	Structured per-tax-component (R-X-2)
tax_total	NUMERIC(15,2) NOT NULL	
total	NUMERIC(15,2) NOT NULL	
package_ref	TEXT NULL	
equipment_ref	TEXT NULL	
period_start, period_end	TIMESTAMP TZ NULL	
sort_order	INTEGER DEFAULT 0	
metadata	JSONB NULL	

Indexes: (parent_type, parent_id, sort_order); (parent_summary_line_id); (subscription_id, period_start) for period queries via parent join. Monthly partitions matching parent. Immutable. P10Y.

Sample data — line items under the Example 1 invoice above (showing a grouped invoice with summary + detail across the 2 wallet types in the Internet & TV group):

id	parent_type	parent_id	line_level	parent_summary_line_id	wallet_type_code	line_type	description	qty	unit_price	subtotal	tax_breakdown	tax_total	total	sort_order
lit_01 HZ...A 1	INVOICE	inv_01 HZQX5K ...	SUMMARY	NULL	WALLET _INTER NET	PACKAGE _CHARGE	Internet 100Mbps bundle	1	4,000	4,000.00	[{"taxCode": "EXCISE", "base": 4000, "tax": 600}, {"taxCode": "VAT", "base": 4600, "tax": 325}]	925.00	4,925.00	10
lit_01 HZ...A 2	INVOICE	inv_01 HZQX5K ...	DETAIL	lit_01 HZ...A 1	WALLET _INTER NET	PACKAGE _CHARGE	Base subscription Sept 1-30	1	3,500	3,500.00	[{"proportional..."}]	809.38	4,309.38	11
lit_01 HZ...A 3	INVOICE	inv_01 HZQX5K ...	DETAIL	lit_01 HZ...A 1	WALLET _INTER NET	PACKAGE _CHARGE	Add-on data bundle 100GB	1	500	500.00	[{"proportional..."}]	115.62	615.62	12
lit_01 HZ...B 1	INVOICE	inv_01 HZQX5K ...	SUMMARY	NULL	WALLET _TV	PACKAGE _CHARGE	TV bundle Premium	1	1,200	1,200.00	[{"taxCode": "EXCISE", "base": 1200, "tax": 180}, {"taxCode": "VAT", "base": 1380, "tax": 97.50}]	277.50	1,477.50	20

id	parent_type	parent_id	line_level	parent_summary_line_id	wallet_type_code	line_type	description	qty	unit_price	subtotal	tax_breakdown	tax_total	total	sort_order
lit_01 HZ...B 2	INVOICE	inv_01 HZQX5K ...	DETAIL	lit_01 HZ...B 1	WALLET_TV	PACKAGE	Base TV Sept 1-30	1	1,000	1,000.00	[...proportional...]	231.25	1,231.25	21
lit_01 HZ...B 3	INVOICE	inv_01 HZQX5K ...	DETAIL	lit_01 HZ...B 1	WALLET_TV	PACKAGE	Pay-per-view 2 events	2	100	200.00	[...proportional...]	46.25	246.25	22

(The corresponding Voice charges for this customer are on the **sibling invoice** inv_01HZQX5K...VOICE with wallet_type_code = WALLET_VOICE, legal number WIK-2026-00012346 — a separate row in the invoice table per Wananchi KE's grouping policy.)

How to read this:

- **SUMMARY lines** (10, 20) show what appears on the customer's summary page — top-level totals per package.
- **DETAIL lines** (11, 12, 21, 22) show consumption breakdown under each summary. They link via parent_summary_line_id.
- All lines on this invoice belong to either WALLET_INTERNET or WALLET_TV — the two wallets in this group. Voice is excluded (lives on the sibling invoice).
- tax_breakdown JSONB per line carries the same structure as the invoice-level tax_summary, just per-line. The invoice's tax_summary is computed by aggregating these.
- sort_order drives the display order on the rendered PDF.
- The invoice's totals (subtotal 5,200 + tax 1,202.50 = total 6,402.50) match the example 1 invoice header above.

invoice_sequence — gap-free sequence allocator for legal invoices

Per R-N-3 and R-N-4. One row per (operator_code, fiscal_year, invoice_type_code). The system increments current_value atomically (SELECT FOR UPDATE → increment → insert invoice) in the same DB transaction as the invoice insert.

Column	Type	Notes
operator_code	TEXT NOT NULL	Part of composite PK
fiscal_year	INTEGER NOT NULL	Part of composite PK. Computed from invoice_type_operator_config.fiscal_year_start_month.
invoice_type_code	TEXT NOT NULL	Part of composite PK. Each legal-document type has its own sequence (CYCLE_POSTPAID, ONE_OFF_INVOICE, TAX_INVOICE, CREDIT_NOTE, DEBIT_NOTE).
current_value	BIGINT NOT NULL DEFAULT 0	The last-allocated sequence number. To allocate next: lock row, current_value = current_value + 1, return the new value.
created_at	TIMESTAMPTZ NOT NULL	When this sequence row was first created (typically at fiscal-year rollover or operator onboarding)
last_allocated_at	TIMESTAMPTZ NOT NULL	When the most-recent allocation happened — useful for ops monitoring.

PRIMARY KEY (operator_code, fiscal_year, invoice_type_code). Change history table (invoice_sequence_history) tracks the rare manual edits operators may make (e.g., correcting a sequence after offline reconciliation — always audited).

Sample data:

operator_code	fiscal_year	invoice_type_code	current_value	last_allocated_at
WANANCHI_KE	2026	CYCLE_POSTPAID	12345	2026-10-01 00:01:32
WANANCHI_KE	2026	TAX_INVOICE	8721	2026-10-10 14:32:18
WANANCHI_KE	2026	CREDIT_NOTE	234	2026-09-28 11:15:00
WANANCHI_KE	2026	DEBIT_NOTE	47	2026-09-15 10:22:34

operator_code	fiscal_year	invoice_type_code	current_value	last_allocated_at
WANANCHI_KE	2026	ONE_OFF_INVOICE	567	2026-09-30 16:42:00
WANANCHI_UG	2026	CYCLE_POSTPAID	5210	2026-10-01 00:02:11
WANANCHI_TZ	2026	CYCLE_POSTPAID	7891	2026-10-01 00:02:45

How to read: each row's `current_value` is the highest sequence number allocated so far. To allocate the next `CYCLE_POSTPAID` invoice number for Wananchi KE in fiscal 2026: lock the row → `current_value` becomes 12346 → write the new invoice with that number → commit transaction. If anything between allocation and commit fails, the allocation is rolled back as part of the transaction.

`pro_forma_sequence` — non-gap-free sequence for pro forma

Per R-N-5. Same row-locking allocation mechanism as `invoice_sequence`, but gaps are acceptable (pro forma is informational, not legal). One row per (`operator_code`, `fiscal_year`). No `invoice_type_code` split since V3 has only one pro forma type today; if a future deployment needs different pro forma types, an `invoice_type_code` column will be added.

Column	Type	Notes
<code>operator_code</code>	TEXT NOT NULL	Part of composite PK
<code>fiscal_year</code>	INTEGER NOT NULL	Part of composite PK
<code>current_value</code>	BIGINT NOT NULL DEFAULT 0	Last-allocated pro forma number
<code>created_at</code>	TIMESTAMPTZ NOT NULL	
<code>last_allocated_at</code>	TIMESTAMPTZ NOT NULL	

PRIMARY KEY (`operator_code`, `fiscal_year`). No change history table (gaps are acceptable; no audit need).

Sample data:

operator_code	fiscal_year	current_value	last_allocated_at
WANANCHI_KE	2026	45892	2026-09-26 03:00:12
WANANCHI_UG	2026	12034	2026-09-26 03:05:44
YAS_SN	2026	8430	2026-09-26 03:15:08

`invoice_sequence_gap` — audit trail for skipped legal invoice numbers

Per R-N-4. Records every case where an `invoice_sequence` number was allocated but the corresponding invoice row could NOT be written successfully (rare — DB constraint violation, line item insert failure, partial transaction failure). The skipped number is preserved here so that operator audits and tax-authority reconciliation can account for every number in the sequence.

This table is **NOT** the same as the `invoice` table. The `invoice` row for that number was never created — only this gap record exists. The gap record carries no customer info, no line items, no amounts. It's purely bookkeeping for "this number was burned and we know why".

Column	Type	Notes
<code>id</code>	TEXT PK	ULID
<code>operator_code</code>	TEXT NOT NULL	Which operator's sequence the gap belongs to
<code>fiscal_year</code>	INTEGER NOT NULL	Which fiscal year
<code>invoice_type_code</code>	TEXT NOT NULL	Which type's sequence (since each has its own)
<code>gap_number</code>	BIGINT NOT NULL	The skipped sequence value (e.g., 12340)
<code>reason_code</code>	TEXT NOT NULL	Why the gap occurred. Examples: LINE_ITEM_INSERT_FAILED, CUSTOMER_SNAPSHOT_FETCH_FAILED, TAX_SERVICE_TIMEOUT_AT_WRITE, DB_CONSTRAINT_VIOLATION, MANUAL_ADMIN_SKIP (extensible)
<code>reason_detail</code>	TEXT NULL	Free-form explanation (stack trace summary, operator note, error message)
<code>recorded_at</code>	TIMESTAMPTZ NOT NULL	When the gap was logged

Column	Type	Notes
status	TEXT NOT NULL DEFAULT 'PENDING_REVIEW'	PENDING_REVIEW (newly logged, awaiting operator action) / RECONCILED_WITH_TAX_AUTHORITY (operator reported to KRA/equivalent and the authority accepted the explanation) / ACKNOWLEDGED_NO_ACTION (operator reviewed; no action needed for this jurisdiction)
resolved_by	TEXT NULL	Admin user ID who resolved the gap
resolved_at	TIMESTAMPTZ NULL	When resolved
resolution_notes	TEXT NULL	What the admin did (e.g., "Reported to KRA Etims via portal on 2026-10-05, reference KRA-2026-Q4-RECONCILE-00482")

Indexes: (operator_code, fiscal_year, invoice_type_code, gap_number) UNIQUE; (status, recorded_at) for the admin review queue. Change history table (invoice_sequence_gap_history) tracks status changes (PENDING_REVIEW → RECONCILED).

Sample data (rare — only populated when an invoice write fails partway):

id	operator_code	fiscal_year	invoice_type_code	gap_number	reason_code	recorded_at	status
gap_01HZ...	WANANCHI_KE	2026	CYCLE_POSTPAID	12340	LINE_ITEM_INSERT_FAILED	2026-10-01 00:01:31	PENDING_REVIEW

How to read this:

- The customer who *would have* received invoice WIK-2026-00012340 has no invoice at that number — the row was never written.
- Sequence number 12341 went to the next customer in the cycle-close batch. The gap is recorded so auditors can confirm: "numbers 12339, 12340 (gap, see audit), 12341 are all accounted for."
- Operator reviews via admin console: looks at the reason_detail, decides if it needs to be reported to the tax authority (Kenya: yes — KRA Etims expects every number to be either an invoice or a documented gap), updates status with notes.
- The original customer whose invoice failed gets a new invoice on the next cycle-close retry — with the NEXT sequence number (12346 or whatever's current at that point). The system does NOT try to reuse 12340.

*_history change history tables — what they look like

Every mutating record (invoice, pro_forma, invoice_type_definition, invoice_type_operator_config, invoice_sequence_gap) has a parallel _history table. Same column structure as the main table, plus 4 audit columns: history_id (ULID), change_timestamp, changed_by (user ID or system actor like BIL-03-CYCLE-SCANNER), change_reason_code. Each time a row in the main table is updated, the OLD values are copied to a new row in the history table. The main table holds the current state; the history table holds the audit trail.

Sample — invoice_history showing one invoice's lifecycle:

history_id	change_timestamp	changed_by	change_reason_code	invoice_id	status	payment_status	paid_amount
h_01...A1	2026-10-01 00:01:32	BIL-03-CYCLE-SCANNER	INVOICE_GENERATED	inv_01HZQX5K...	GENERATED	UNPAID	0.00
h_01...A2	2026-10-01 03:14:08	NOTIFICATION-MODULE	NOTIFICATION_DELIVERED	inv_01HZQX5K...	PENDING	UNPAID	0.00
h_01...A3	2026-10-10 14:30:00	BIL-01-PAYMENT-HANDLER	PAYMENT_APPLIED_PARTIAL	inv_01HZQX5K...	PENDING	PARTIALLY_PAID	3500.00
h_01...A4	2026-10-22 09:15:45	BIL-01-PAYMENT-HANDLER	PAYMENT_APPLIED_FULL	inv_01HZQX5K...	PAID	PAID	7018.00

How to read this:

- Four rows = four changes to the same invoice over its lifecycle.
- Row 1: invoice created by the cycle scanner.
- Row 2: notification module confirmed customer received it, status moved to PENDING.
- Row 3: customer made a partial payment of 3,500 KES.
- Row 4: customer paid the remaining 3,518 KES, invoice now PAID.

- A compliance auditor or care agent can read this table to reconstruct exactly what happened, when, and who/what caused each change. The current state of the invoice (row 4's values) is what's in the main invoice table.

The same pattern applies to `pro_forma_history` (tracks GENERATED → PRESENTED → SUPERSEDED/FULFILLED transitions), `invoice_type_definition_history` (tracks catalog edits — rare, audited), and `invoice_type_operator_config_history` (tracks per-operator config changes — e.g., when an admin changes due-date grace days).

Generator framework — developer guide

BIL-02 Core provides a **generator framework** that satellite DDs (and future deployments) plug into. This section explains how it works concretely for a developer who will write a generator class.

What the framework provides

The framework consists of four pieces that live in BIL-02 Core:

1. **InvoiceGenerator interface** — what every generator must implement.
2. **InvoiceGenerationService** — the central entry point that receives triggers and dispatches to the right generator.
3. **InvoiceGeneratorRegistry** — at module startup, scans for classes implementing **InvoiceGenerator** and registers them indexed by `getTypeCode()`. Looks them up by `invoice_type_definition.generator_class_ref`.
4. **Shared services** that generators use to do their work: **InvoiceSequenceService**, **InvoiceLineBuilder**, **InvoiceWriter**, **ProFormaWriter**, **CustomerSnapshotService**, **InvoiceEventPublisher**.

Each generator is responsible for ONE invoice type. The framework dispatches to it; the generator does its specific work; the framework's shared services handle the cross-cutting concerns (sequence, writing, events).

The InvoiceGenerator interface

```
public interface InvoiceGenerator {

    /**
     * Returns the invoice_type_code this generator handles.
     * Must match a row in invoice_type_definition.type_code.
     * Used by InvoiceGeneratorRegistry to dispatch.
     */
    String getTypeCode();

    /**
     * Produces zero or more invoices for the given context.
     * Returns a list because grouping policies may produce N invoices per trigger
     * (e.g., Wananchi KE WALLET dimension with 2 groups produces 2 invoices per cycle).
     * Returns empty list when the trigger results in no invoices (e.g., zero-charge cycle).
     * Returns existing invoice(s) when idempotency check finds previous match.
     */
    List<Invoice> generate(InvoiceGenerationContext context);
}
```

In Laravel/PHP the equivalent is an interface with the same contract; the rest of this section uses Java syntax for concreteness.

How a generator gets discovered and called

The lifecycle has three phases:

Phase 1 — Module startup (one-time at boot):

The Spring container scans the classpath for `@Component`-annotated classes implementing **InvoiceGenerator**. For each one found, it calls `getTypeCode()` and registers the bean in **InvoiceGeneratorRegistry** indexed by that code. After startup, the registry has a map like:

```
"CYCLE_POSTPAID" → CyclePostpaidInvoiceGenerator instance
"ONE_OFF_INVOICE" → OneOffInvoiceGenerator instance
"PRO_FORMA_CYCLE" → ProFormaCycleGenerator instance
"TAX_INVOICE" → TaxInvoiceGenerator instance
"CREDIT_NOTE" → CreditNoteGenerator instance
"DEBIT_NOTE" → DebitNoteGenerator instance
```

The same generator classes are listed in `invoice_type_definition.generator_class_ref`. If a row in the catalog references a generator class that's NOT registered at startup, the system logs a configuration error and refuses to generate that type — protecting against half-deployed code.

Phase 2 — Trigger arrives:

A Kafka event consumer (e.g., `CycleClosedConsumer` for cycle invoices, `PaymentAppliedConsumer` for tax invoices), a scheduled scanner (e.g., `ProFormaScanner` for daily pro forma generation), or an admin REST endpoint receives a trigger. It builds an `InvoiceGenerationContext` and calls:

```
List<Invoice> invoices = invoiceGenerationService.generate(context);
```

`InvoiceGenerationService.generate()` looks up the right generator by `context.getInvoiceTypeCode()`, delegates, and returns the result.

Phase 3 — Generator execution:

The generator's `generate()` method is called. It uses framework services to do its work (see below). The returned `Invoice` objects are already written to the database and have lifecycle events published.

The `InvoiceGenerationContext` parameter object

Every generator receives this context. It contains everything the generator needs to do its job without making redundant lookups.

Field	Type	Source	What it's for
<code>invoiceTypeCode</code>	String	Set by the trigger consumer	Which generator to dispatch to
<code>triggeringEventRef</code>	String	Event ID or scheduler tick ID	Idempotency key
<code>subscriptionId</code>	String	From trigger event	Which subscription this invoice belongs to
<code>subscriptionSnapshot</code>	SubscriptionMaster	Loaded from SUB-LM-01 at trigger time	Billing mode, customer location, currency, current cycle bounds — read-only snapshot
<code>operatorCode</code>	String	From subscription snapshot	Identifies operator for config lookup
<code>operatorConfig</code>	InvoiceTypeOperatorConfig	Loaded from BIL-02 Core repo at trigger time	Per-operator settings (grace days, format, signing endpoint, etc.)
<code>groupingPolicy</code>	List<InvoiceGroupingConfig>	Loaded at trigger time	The operator's policy for this invoice type (groups + dimension)
<code>cycleWindow</code>	TimeWindow	For cycle invoices: from <code>CycleClosed</code> event; for one-off: null	Period start/end for line items
<code>chargesFromBilling</code>	ChargeComputationResponse	Pre-loaded by trigger consumer from BIL-01 Charging Engine	The actual charges to put on the invoice (already pre-computed by BIL-01 before the generator runs)
<code>paymentRef</code>	String	For tax invoices: the BIL-01 <code>PaymentApplied</code> event ID	Links tax invoice to its triggering payment
<code>originalInvoiceId</code>	String	For credit/debit notes: the invoice being adjusted	Linkage
<code>adjustmentRequest</code>	InvoiceAdjustment	For credit/debit notes from BIL-02-ADJ-01	The adjustment that triggered this note
<code>adminContext</code>	AdminActionContext	For admin-triggered generation: who, when, why	Audit trail

The trigger consumer builds the context once before calling the service. The generator doesn't have to re-fetch any of this.

Shared services every generator uses

A generator's body is mostly orchestration of these services:

Service	Purpose	Method examples
<code>InvoiceSequenceService</code>	Allocates gap-free legal invoice numbers (or non-gap-free pro forma numbers). Handles atomic increment and gap recording.	<code>allocateLegalNumber(operator, fiscalYear, typeCode, format)</code> returns "WIK-2026-00012345". Throws <code>SequenceAllocationException</code> if write fails — the framework records a gap.
<code>InvoiceLineBuilder</code>	Transforms BIL-01-returned charges + <code>TaxService</code> -computed taxes into invoice line items (SUMMARY + DETAIL two-level structure). Handles grouping by dimension.	<code>build(invoice, charges, groupingDimension, groupKeyValues)</code> returns <code>List<InvoiceLineItem></code> .
<code>CustomerSnapshotService</code>	Reads from CRM/customer service and produces the immutable <code>customer_snapshot</code> JSONB.	<code>captureSnapshot(customerId, snapshotTimestamp)</code> returns <code>Map<String, Object></code> ready to store. Throws <code>CrmsUnavailableException</code> on failure.

Service	Purpose	Method examples
InvoiceWriter	Writes invoice + line items + customer snapshot in a single DB transaction. Enforces uniqueness on (triggering_event_ref, invoice_type_code, grouping_config_ref).	write(invoice, lineItems) returns the persisted invoice.
ProFormaWriter	Same as InvoiceWriter but for the pro_forma table.	write(proForma, lineItems)
InvoiceEventPublisher	Emits InvoiceIssued / ProFormaIssued / TaxInvoiceIssued Kafka events with full payload (invoice + line items + customer_snapshot).	publishInvoiceIssued(invoice, lineItems)
TaxServiceClient	REST client to PLM-CFG-02 Tax Configuration.	computeTax(lineItem, customerLocation, packageRef) returns TaxComputationResult.
IdempotencyChecker	Looks up existing invoice by (triggering_event_ref, invoice_type_code, grouping_config_ref) to make Kafka retries safe.	findExisting(eventRef, typeCode, groupConfigRef) returns Optional<Invoice>.

Skeleton — the canonical generator shape

The actual generator classes (CyclePostpaidInvoiceGenerator, OneOffInvoiceGenerator, ProFormaCycleGenerator, CreditNoteGenerator, DebitNoteGenerator) and their per-class flows live in **BIL-02-GEN-01 Invoice Generation**. Core only defines the framework shape; the satellite owns the concrete implementations.

Every generator follows the same canonical structure when using the framework services Core provides. The skeleton below is a shape sketch — not a real class — illustrating how a generator orchestrates the shared services:

```
@Component
@Transactional
public class SomeInvoiceGenerator implements InvoiceGenerator {

    // Inject framework services from BIL-02 Core
    private final InvoiceSequenceService sequenceService;
    private final InvoiceLineBuilder lineBuilder;
    private final CustomerSnapshotService snapshotService;
    private final InvoiceWriter invoiceWriter;
    private final InvoiceEventPublisher eventPublisher;
    private final IdempotencyChecker idempotency;

    @Override
    public String getTypeCode() { return "..."; }

    @Override
    public List<Invoice> generate(InvoiceGenerationContext ctx) {
        // 1. Subscription state gate (skip if PENDING)
        // 2. Capture customer_snapshot via snapshotService
        // 3. For each group in ctx.getGroupingPolicy():
        //    a. Idempotency check via idempotency.findExisting(...)
        //    b. Filter charges/inputs for this group
        //    c. Build line items via lineBuilder.build(...)
        //    d. Build invoice header (set fields from ctx + snapshot + group)
        //    e. Allocate sequence number via sequenceService
        //    f. Write atomically via invoiceWriter.write(invoice, lines)
        //    g. Publish event via eventPublisher
        // 4. Return list of persisted invoices
    }
}
```

Each of the 5 concrete generators in BIL-02-GEN-01 Invoice Generation specializes this shape with type-specific logic: cycle generator fetches charges from BIL-01 Charging Engine; one-off generator uses the fee descriptor directly from the event; pro forma generator runs from a scanner with projected charges; credit/debit note generators receive their inputs from BIL-02-ADJ-01 Invoice Adjustments. See that DD for the full step-by-step flow of each generator.

How a future deployment plugs in a new generator

A future deployment wanting to add a new invoice type does these steps:

1. **Write the generator class** — implement InvoiceGenerator, annotate with @Component. Follow the canonical shape sketched above and the worked examples in BIL-02-GEN-01 Invoice Generation. Use existing framework services for sequence, line items, snapshot, write, events.
2. **Insert a row in invoice_type_definition** — register the new type code, set generator_class_ref to the fully-qualified class name of the new generator, choose legal_nature, trigger_type, requires_signing,

legal_sequence_required, customer_deliverable, active.

3. **Insert rows in invoice_type_operator_config** — one per operator that will use the new type, with their specific settings (grace days, format, signing endpoint if applicable).
4. **Insert rows in invoice_grouping_config** — define how charges are grouped for the new type per operator (which dimension, which groups). For most non-cycle types (one-offs, tax invoices, notes), often a single SINGLE-dimension group is enough.
5. **Add a trigger source** — either subscribe to an existing Kafka topic, register a new scheduled scanner, or expose a new admin endpoint. The trigger source builds the InvoiceGenerationContext and calls InvoiceGenerationService.generate(context).

That's the whole extension surface. No changes to:

- invoice table schema (the new type uses the same columns)
- invoice_line_item table schema
- InvoiceSequenceService (just allocates per new type_code automatically)
- InvoiceWriter, CustomerSnapshotService, InvoiceEventPublisher (same code, same events)
- Other generators (each is independent)
- DD_NOT-01 Notification Service (it already handles the standard event payload; only needs new template if the operator wants a different layout)

Events (Kafka) — core only

Emitted by core

Topic	When	Consumers
sophix.billing.invoice.issued	Every legal invoice creation (called by satellite generators)	BIL-01 Charging Engine, DD_NOT-01 Notification Service, CVM, analytics
sophix.billing.invoice.paymentapplied	Payment applied to invoice (called by BIL-02-GEN-01 on PaymentApplied consumer)	BIL-01 Charging Engine, DD_NOT-01 Notification Service
sophix.billing.invoice.cancelled	Invoice cancelled (admin or bulk)	BIL-01 Charging Engine, DD_NOT-01 Notification Service, BIL-04 Dunning Engine
sophix.billing.proforma.issued	Every pro forma creation (called by BIL-02-GEN-01)	DD_NOT-01 Notification Service, CVM
sophix.billing.invoice.sequencegap	Rare — gap detected	operator alert
sophix.billing.invoice.skippedduetopen dingtoolong	Stuck Subscription PENDING_* >4h	operator alert

Sample sophix.billing.invoice.issued event payload:

```
{
  "eventId": "evt_01HZQX5K3M9P0V8YQ6",
  "eventType": "InvoiceIssued",
  "eventTimestamp": "2026-10-01T00:01:32Z",
  "invoice": {
    "id": "inv_01HZQX5K3M9P0V8YQ6",
    "invoiceTypeCode": "CYCLE_POSTPAID",
    "legalInvoiceNumber": "WIK-2026-00012345",
    "subscriptionId": "sub_01HVR3T2W5K8",
    "customerId": "cust_01HVR3T1A9F2",
    "operatorCode": "WANANCHI_KE",
    "currency": "KES",
    "billingMode": "POSTPAID",
    "groupingConfigRef": "gc_01...A1",
    "groupingDimension": "WALLET",
    "groupName": "Internet & TV",
    "groupingKeyValues": ["WALLET_INTERNET", "WALLET_TV"],
    "customerSnapshot": {
      "fullName": "John Mwangi Kamau",
      "billingAddress": {
        "line1": "Plot 45, Ngong Road",
        "line2": "Lavington",
        "city": "Nairobi",
        "region": "Nairobi County",
        "postalCode": "00100",
        "countryCode": "KE"
      }
    },
    "email": "john.kamau@example.co.ke",
    "phone": "+254712345678",
  }
}
```

```

    "languagePreference": "EN",
    "taxIdentifier": {"type": "KRA_PIN", "value": "A001234567Z"},
    "customerType": "INDIVIDUAL",
    "snapshotTakenAt": "2026-10-01T00:01:30Z"
  },
  "issueDate": "2026-10-01T00:01:32Z",
  "dueDate": "2026-10-15T23:59:59Z",
  "subtotalAmount": 5200.00,
  "taxAmountTotal": 1202.50,
  "taxSummary": [
    {"taxCode": "EXCISE", "taxName": "Excise Duty", "totalBaseAmount": 5200.00, "totalTaxAmount": 780.00},
    {"taxCode": "VAT", "taxName": "Value Added Tax", "totalBaseAmount": 5980.00, "totalTaxAmount": 422.50}
  ],
  "totalAmount": 6402.50,
  "periodStart": "2026-09-01T00:00:00Z",
  "periodEnd": "2026-09-30T23:59:59Z",
  "lineItems": [
    {
      "id": "lit_01HZ..A1",
      "lineLevel": "SUMMARY",
      "walletTypeCode": "WALLET_INTERNET",
      "lineType": "PACKAGE_CHARGE",
      "description": "Internet 100Mbps bundle",
      "quantity": 1,
      "unitPrice": 4000.00,
      "subtotal": 4000.00,
      "taxBreakdown": [
        {"taxCode": "EXCISE", "ratePercent": 15, "baseAmount": 4000, "taxAmount": 600},
        {"taxCode": "VAT", "ratePercent": 16, "baseAmount": 4600, "taxAmount": 325}
      ],
      "taxTotal": 925.00,
      "total": 4925.00,
      "sortOrder": 10
    },
    "... (other line items)"
  ]
},
"operatorContext": {
  "operatorCode": "WANANCHI_KE",
  "displayName": "Wananchi Group – Kenya"
}
}

```

The notification service has everything it needs in this payload to render the PDF, compose the email body, and dispatch — without further calls to BIL-02 or CRM or Subscription. The only thing notification fetches separately is the customer's channel preferences and the operator's current template assets.

Consumed by core

Topic	From	Action
sophix.notification.invoice.presented	notification module	Transition invoice/pro forma to PRESENTED

Satellites consume their own events (BIL-02-STA-01 consumes PaymentApplied for PAID transition; BIL-02-GEN-01 consumes CycleClosed, OneOffChargeRequested, etc.).

Notification contract — what BIL-02 sends vs what DD_NOT-01 Notification Service fetches

When BIL-02 emits InvoiceIssued (or ProFormaIssued or TaxInvoiceIssued), DD_NOT-01 Notification Service consumes the event and is responsible for rendering the PDF and dispatching it through the customer's preferred channels. The boundary between what's in the event vs what notification fetches separately is deliberate.

What the event payload carries (everything needed to render the PDF):

Section	Content	Source
Invoice header	invoice_id, type, legal number, operator, currency, billing_mode, issue/due dates, totals	Invoice table
Customer identification	full_name, billing_address, email, phone, language_preference, tax_identifier, customer_type, business_registration_number (if BUSINESS)	customer_snapshot JSONB on invoice
Line items	All SUMMARY + DETAIL lines with descriptions, amounts, periods, packages, wallet types	Invoice line items
Tax breakdown	Structured per-tax-component totals (e.g., Excise + VAT separately)	tax_summary JSONB

Section	Content	Source
Grouping context	grouping_dimension, group_name (from config at issuance), grouping_key_values	Invoice header + config snapshot
Operator context	Operator code (used by notification to load operator-specific branding/template)	Invoice header

What notification fetches separately (delivery preferences and template assets):

Need	Source	Why not on invoice
Customer's channel preferences (email opt-in, SMS opt-in, push opt-in, portal-only)	DD_NOT-01 preference store	These can change without invalidating the historical invoice — they're delivery state, not invoice content
Customer's quiet-hours preference (don't send 22:00–07:00)	DD_NOT-01 preference store	Same — delivery metadata
Operator template version + branding assets (logos, colors, layout)	DD_NOT-01 template registry	Templates evolve; the right template version was captured on the invoice at issuance and used at re-render time
Delivery state (sent, delivered, opened, bounced, retried)	DD_NOT-01 delivery log	Mutable, not part of the invoice
Customer's email/SMS deliverability state (bounced, blacklisted)	DD_NOT-01 deliverability cache	Mutable

Why this split:

- The invoice is a legal document; its content must be immutable and self-contained. Customer details captured at issuance are part of the document and protected by the change history table (invoice_history never modifies customer_snapshot).
- Notification preferences and delivery state are operational — they change daily, are mutable, and shouldn't pollute the legal document store.
- Re-rendering an invoice years later: notification reads the customer_snapshot from the invoice (legal customer state at the time), combines it with the operator's current template version (recorded at original render), and produces an identical PDF — without ever needing to call CRM or Subscription master.

Tax invoice nuance: When BIL-02-TAX-01 produces a tax invoice, it captures a **fresh** customer_snapshot at payment moment, not the snapshot from the original cycle invoice. The tax invoice reflects customer state at tax declaration time. If the customer changed their tax identifier or address between cycle and payment, the tax invoice shows the new state — which is what the tax authority expects.

APIs (core only)

GET /api/invoices/{id}	← legal invoice with line items
GET /api/invoices/by-legal-number/{legalNumber}	← lookup
GET /api/invoices?subscription_id={id}	← invoice history
GET /api/invoices?account_id={id}	← account invoice history
GET /api/invoices?customer_id={id}	← customer-level search/grouping
GET /api/invoices?operator={code}&status={s}	← admin listing
GET /api/proforma/{id}	← pro forma by id
GET /api/proforma?subscription_id={id}	← pro forma history
GET /api/customers/me/documents	← all customer docs
GET /api/invoice-types	← catalog admin
POST /api/invoice-types	← add type (extensibility)
GET /api/invoice-types/{typeCode}/operator-config/{op}	← per-operator config
POST /api/invoice-types/{typeCode}/operator-config	← update per-operator config
GET /api/invoice-sequence-gaps	← gap review queue
POST /api/invoice-sequence-gaps/{id}/resolve	← mark resolved

Satellite endpoints (download, retrigger, adjustments, PTP, dispute, bulk reversal, tax signing retry) live in their respective DDs.

Cross-DD coordination

Touched	What core owns	What the other DD owns
BIL-02-GEN-01 Invoice Generation	Provides type catalog, sequence service, invoice/pro forma writers, multi-key line item rendering (grouped or separate per dimension). Receives generator class implementations from GEN-01.	Owens all generators (cycle, one-off, pro forma, credit/debit note), idempotency, bulk reversal

Touched	What core owns	What the other DD owns
BIL-02-TAX-01 Tax Invoice & Gateway	Provides invoice writer with signing-state fields. Holds the TAX_INVOICE type catalog row + per-operator config.	Owens tax invoice signing flow, Tax Gateway integration, retry scanner, failure queue
BIL-02-ADJ-01 Invoice Adjustments	Provides invoice writer with original_invoice_id + credit/debit note type catalog.	Owens adjustment entity, approval workflow, refund routing
BIL-02-READ-01 Invoice Read API	Provides invoice read API + line item joins.	Owens PDF streaming, Redis caching, access control, download audit log
BIL-02-STA-01 Status Management & Recovery	Provides invoice lifecycle state machine + transition triggers.	Owens daily scanners (overdue, pre-due), PTP, dispute, late fee, pay-to-restore
SUB-WF-FRAMEWORK-01 Subscription Workflow Framework	Follows the scheduled-scanner + synchronous-service pattern.	Owens the framework
SUB-LM-01 Subscription Lifecycle Management	Writes current_cycle_pro_forma_id via put-master-fields; reads master fields (billing_mode, customer_location, currency, current_cycle_*).	Owens Subscription master
BIL-01 Charging Engine	Calls POST /api/billing/charges/compute for cycle and tax invoice line items. Emits InvoiceIssued, InvoicePaymentApplied, InvoiceCancelled consumed by BIL-01 for outstanding balance recomputation.	Owens charging logic, payment recording, outstanding balance
BIL-03 Cycle Close	Indirect — BIL-02-GEN-01 Invoice Generation consumes CycleClosed.	Owens cycle boundary detection
BIL-04 Dunning Engine	Indirect — BIL-02-STA-01 Status Management & Recovery emits InvoiceOverdue consumed by BIL-04.	Owens dunning state machine
BIL-05 Wallet and Topup	Indirect — BIL-02-TAX-01 Tax Invoice & Gateway consumes WalletDebited filtered for cycle debit.	Owens wallet
PLM-CFG-01 Package Catalog	When grouping_dimension=PACKAGE, validates group_key_values against package catalog at policy save time.	Owens package definitions
PLM-CFG-02 Tax Configuration	Calls TaxService.computeTax per line at generation time (via BIL-02-GEN-01 Invoice Generation); stores returned breakdown.	Owens tax rules (which taxes apply, cascade, rates, exemptions)
PLM-CFG-03 Wallet Configuration	Stores wallet_type_code FK on invoice header, pro forma header, and line items. When grouping_dimension=WALLET, validates group_key_values against wallet catalog at policy save time.	Owens wallet type catalog (operator-configurable wallet structures)
PLM-CFG-04 Service Catalog	When grouping_dimension=SERVICE_CATEGORY, validates group_key_values against service catalog at policy save time.	Owens service category definitions
CRM / Customer Service	Calls customer service to read customer details for customer_snapshot capture at invoice generation. Snapshot is then frozen on the invoice.	Owens live customer master (name, address, contact, language, tax ID)
DD_NOT-01 Notification Service	Emits InvoiceIssued, ProFormaIssued, TaxInvoiceIssued, InvoicePaymentApplied, InvoiceCancelled with full payload (invoice + line items + customer_snapshot — see "Notification contract" above). Consumes InvoicePresented for lifecycle transition.	Owens templates, PDF rendering, channel dispatch, customer preferences, delivery state tracking

Dependencies

Dependency	Owner
SUB-WF-FRAMEWORK-01 Subscription Workflow Framework	subscription team
SUB-LM-01 Subscription Lifecycle Management	subscription team
BIL-01 Charging Engine	billing team
BIL-03 Cycle Close	billing team
BIL-04 Dunning Engine	billing team
BIL-05 Wallet and Topup	billing team
PLM-CFG-01 Package Catalog	catalog team

Dependency	Owner
PLM-CFG-02 Tax Configuration	catalog team
PLM-CFG-03 Wallet Configuration	catalog team
PLM-CFG-04 Service Catalog	catalog team
CRM / Customer Service module	CRM team
DD_NOT-01 Notification Service	notification team
FOUNDATION_DB.md, FOUNDATION_KAFKA.md, FOUNDATION_AUTH.md	infrastructure team
BIL-02-GEN-01 Invoice Generation (satellite)	billing team
BIL-02-TAX-01 Tax Invoice & Gateway (satellite)	billing team
BIL-02-ADJ-01 Invoice Adjustments (satellite)	billing team
BIL-02-READ-01 Invoice Read API (satellite)	billing team
BIL-02-STA-01 Status Management & Recovery (satellite)	billing team

(Implementation status is tracked in the project tracker, not in the DD body, as DD content should be stable across project phases. The infrastructure foundation files need amendment for the new **billing-invoicing** module and the **INVOICE_ADMIN** / **INVOICE_OPERATOR** roles — see those documents for details.)

Operator-configurable capabilities

Every operator-specific behavior in BIL-02 Core is **configurable** via `invoice_type_operator_config` and `invoice_grouping_config`. Operators turn capabilities on or off and tune parameters per their deployment. The system supports all the following without code change:

Capability	Where configured	Notes
Per (operator, invoice_type) enablement	<code>invoice_type_operator_config.enabled</code>	Operator decides which invoice types they use
Tax invoice generation	<code>invoice_type_operator_config.enabled</code> for <code>TAX_INVOICE</code> row + signing endpoint	Pluggable Tax GW per operator. The tax-invoice capability is fully built; operators enable it where their tax regulator requires it.
Tax signing service implementation	<code>signing_service_implementation_ref</code>	Per-operator signer class (e.g., Wananchi Autosend → KRA Etimes for Kenya; operators in other jurisdictions plug in their own implementation)
Due-date grace days	<code>due_date_grace_days</code>	Per operator + invoice type
Pre-due reminder window	<code>pre_due_reminder_window_days</code>	Per operator
Pre-cycle pro forma window	<code>pre_cycle_pro_forma_window_days</code>	Per operator
Legal number format	<code>legal_number_format</code>	Templated per operator (e.g., <code>WIK-{YYYY}-{NNNNNNNN}</code>)
Fiscal year start month	<code>fiscal_year_start_month</code>	Per operator
Grouping policy	<code>invoice_grouping_config</code> rows	Dimension (WALLET / PACKAGE / SERVICE_CATEGORY / SINGLE) + groups per operator + invoice type
Cold storage age	<code>cold_storage_age_days</code>	PDF lifecycle threshold

The sample data in this DD shows one possible V3-launch configuration. The actual values are decisions made by each operator at deployment time and changed through the admin console.

Subscription / wallet model (confirmed)

A Subscription holds a package or bundle. The package or bundle itself owns one or more wallets (per PLM-CFG-03 Wallet Configuration catalog). For Wananchi, a typical bundle holds three wallets (Internet, TV, Voice); the customer's Subscription thus has three wallets associated through its bundle. For Yas SN, packages have a single wallet (or zero, depending on the operator's configuration — PREPAID customers without a wallet pay per cycle directly via BIL-05 Wallet and Topup).

BIL-02 Core uses `wallet_type_code` per line item and per invoice (when `grouping_dimension=WALLET`) to identify which wallet a charge belongs to. The grouping policy then determines how those wallet-tagged charges are distributed across invoices. BIL-05 Wallet and Topup owns the wallet entity itself, including the relationship between Subscription → bundle/package → wallets.

B — Current TZ

TZ Sophix invoicing today:

- Embedded in billing-engine cron job; no dedicated module.
- POSTPAID cycle invoices generated at cycle close. Wananchi-specific.
- Single-tax-amount flat scalar on invoice; KE Excise + VAT cascade hardcoded approximately.
- Adjustments exist in a separate admin tool with disconnected number sequence.
- No structured line items; single-level summary stored.
- No change history tables (no parallel record of who changed what when).
- No invoice type catalog — adding a type means code change.
- No per-operator dual-wallet config.

C — Gap & bridge

Changes from TZ → V3 (core scope only)

Topic	TZ Sophix	V3 core
Architecture	Cron-embedded	Dedicated module + satellite DDs
Invoice types	Hardcoded POSTPAID	Extensible catalog with V3 launch types + future extensibility
Multi-tax	Single flat scalar	Structured per-line + invoice-level tax summary supporting arbitrary tax stacks
Grouping	Not configurable (one invoice per customer per cycle)	Per-operator policy with selectable dimension (WALLET / PACKAGE / SERVICE_CATEGORY / SINGLE); supports hybrid groupings
Customer state on invoice	Live lookup from CRM at render time	Immutable customer_snapshot captured at issuance (compliance-safe)
Pro forma	Informal	First-class entity with own table, sequence, lifecycle, customer snapshot
Line items	Single summary	Two-level (summary + detail) with key tagging per dimension (wallet, package, category)
Legal numbering	Single sequence	Per-(operator, fiscal_year, type), gap-free
Lifecycle	Implicit	5-state Wananchi-aligned (GENERATED → PENDING → PAID/OVERDUE → CANCELLED)
Change history	Application logs only	Change history tables on every mutating record + immutable line items + Kafka events + P10Y retention
Notification interface	Tight coupling (notification calls back to invoice + CRM)	Self-contained event payload with customer_snapshot — notification renders without callbacks

What's NEW in V3 core (not in TZ)

1. Dedicated billing-invoicing module on cluster C as a chassis for all invoicing capabilities.
2. Extensible invoice type catalog (config-driven, generators pluggable).
3. Pro forma as first-class entity in separate table.
4. Two-level line items (summary + detail) with operator-configurable grouping by wallet, package, or service category.
5. **Selectable grouping dimension** — operators choose whether grouping is by wallet, package, service category, or none.
6. **Hybrid grouping** — operators can mix grouped + separate (e.g., 2 wallets together + 1 separate).
7. Multi-tax summary structure (per-line + invoice-level JSONB) supporting arbitrary tax stacks.
8. Per-(operator, fiscal_year, type) gap-free legal invoice numbering.
9. **Customer snapshot** on every invoice and pro forma — immutable historical record for legal compliance and self-contained PDF rendering.
10. Change history tables (*_history) on every mutating record + immutable line items + P10Y retention.

11. 5-state lifecycle aligned with Wananchi workshop.
12. Shared infrastructure that 5 satellite DDs plug into without duplicating code.
13. Documented notification contract — event payload contains everything for PDF rendering; notification fetches only delivery preferences separately.